

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC MỞ THÀNH PHỐ HỒ CHÍ MINH**



ĐÀO NGUYỄN HÙNG

**ỨNG DỤNG RETRIEVAL AUGEMENTED GENERATION XÂY DỰNG
HỆ THỐNG TÁC TỬ AI TRONG NÔNG NGHIỆP THÔNG MINH**

**ĐỒ ÁN NGÀNH
NGÀNH KHOA HỌC MÁY TÍNH**

TP. HỒ CHÍ MINH, 2025

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC MỞ THÀNH PHỐ HỒ CHÍ MINH



ĐÀO NGUYỄN HÙNG

ỨNG DỤNG RETRIEVAL AUGMENTED GENERATION XÂY DỰNG
HỆ THỐNG TÁC TỬ AI TRONG NÔNG NGHIỆP THÔNG MINH

Mã số sinh viên: 2251010032

ĐỒ ÁN NGÀNH
NGÀNH KHOA HỌC MÁY TÍNH

Giảng viên hướng dẫn: ThS. Hồ Hương Thiên

TP. HỒ CHÍ MINH, 2025

LỜI CẢM ƠN

Lời đầu tiên với lòng biết sâu sắc, em xin gửi lời cảm ơn chân thành đến tất cả mọi người đã hỗ trợ và tạo điều kiện thuận lợi để em có thể hoàn thành đồ án “**Ứng dụng Retrieval-Augmented Generation (RAG) xây dựng tác tử AI trong Nông nghiệp thông minh**”.

Trước hết, em xin bày tỏ lòng tri ân sâu sắc nhất đến thầy **Hồ Hướng Thiên**, người đã tận tình hướng dẫn, truyền đạt kiến thức, dành thời gian để góp ý, chỉnh sửa và động viên em trong suốt quá trình thực hiện đề tài. Những định hướng và lời khuyên chuyên môn của thầy không chỉ giúp em vượt qua khó khăn về mặt kỹ thuật mà còn mở rộng tầm nhìn về tiềm năng ứng dụng của trí tuệ nhân tạo trong Nông nghiệp.

Em xin chân thành cảm ơn **Khoa Công nghệ Thông tin** đã tạo môi trường học tập và nghiên cứu lý tưởng, cung cấp đầy đủ cơ sở vật chất, tài liệu và điều kiện tốt nhất để em có thể hoàn thành chương trình học và đề tài đồ án này.

Em cũng xin gửi lời cảm ơn đến quý thầy cô trong **Khoa Công nghệ Thông tin** đã cung cấp những kiến thức nền tảng vững chắc, là hành trang quan trọng để em tiếp cận và giải quyết các vấn đề của đề tài.

Cuối cùng, em xin cảm ơn gia đình, bạn bè và những người thân yêu đã luôn là nguồn động viên, ủng hộ tinh thần to lớn trong suốt quá trình học tập và hoàn thành đồ án.

Mặc dù đã có nhiều cố gắng, đề tài chắc chắn không tránh khỏi những thiếu sót. Em rất mong được những ý kiến đóng góp quý báu của quý thầy cô và bạn đọc để đề tài được hoàn thiện hơn trong tương lai.

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

MỤC LỤC

DANH MỤC TỪ VIẾT TẮT	6
DANH MỤC HÌNH ẢNH	7
DANH MỤC BẢNG	9
MỞ ĐẦU.....	1
Chương 1. GIỚI THIỆU ĐỀ TÀI.....	2
1.1. Lý do chọn đề tài	2
1.2. Mục tiêu nghiên cứu	2
1.3. Đối tượng nghiên cứu	2
1.4. Bố cục đề tài	3
Chương 2. CÁC CÔNG NGHỆ VÀ CƠ SỞ LÝ THUYẾT	4
2.1. Trí tuệ nhân tạo tạo sinh và mô hình Ngôn ngữ lớn (LLM).....	4
2.1.1. Trí tuệ nhân tạo tạo sinh (Generative AI).....	4
2.1.2. Mô hình Ngôn ngữ lớn (Large Language Model – LLM).....	5
2.1.3. Vấn đề “Ảo giác” (Hallucination) trong LLM	6
2.2. Vector Embedding và Cơ sở dữ liệu Vector (Vector Store)	8
2.2.1. Vector Embedding	8
2.2.2. Cơ sở dữ liệu Vector (Vector Store).....	9
2.3. Kiến trúc Retrieval-Augmented Generation (RAG)	9
2.3.1. Giai đoạn 1: Lập chỉ mục (Indexing).....	10
2.3.2. Giai đoạn 2: Truy xuất (Retrieval).....	11
2.3.3. Giai đoạn 3: Tạo sinh tăng cường (Augmented Generation)	12
2.4. Tác tử AI (AI Agent)	12
2.4.1. Khái niệm Tác tử AI	12
2.4.2. Các loại Tác tử AI.....	13

2.5.	LangChain	16
2.5.1.	Khai niệm LangChain.....	16
2.5.2.	Nguyên lý hoạt động và mục tiêu	16
Chương 3. XÂY DỰNG HỆ THỐNG TÁC TỬ AI TRONG NÔNG NGHIỆP THÔNG MINH ỨNG DỤNG RAG		18
3.1.	Giới thiệu hệ thống	18
3.2.	Các chức năng chính của hệ thống	18
3.3.	Kiến trúc hệ thống	19
3.4.	Luồng hoạt động của hệ thống	20
3.4.1.	Luồng xử lý dữ liệu (Xây dựng Cơ sở tri thức).....	20
3.4.2.	Luồng trả lời câu hỏi (Tương tác thời gian thực)	20
3.5.	Phân tích hệ thống	22
3.5.1.	Lược đồ use case hệ thống AgriBot.....	22
3.5.2.	Đặc tả chức năng use case đăng ký người dùng	22
3.5.3.	Đặc tả chức năng use case đăng nhập người dùng	23
3.5.4.	Đặc tả chức năng use case đưa ra câu hỏi.....	24
3.5.5.	Đặc tả chức năng use case trả lời câu hỏi	25
3.5.6.	Đặc tả chức năng use case tiếp tục ngữ cảnh câu hỏi	26
3.5.7.	Đặc tả chức năng use case tìm kiếm hội thoại.....	26
3.5.8.	Đặc tả chức năng use case xóa hội thoại	27
3.5.9.	Đặc tả chức năng use case cập nhật cơ sở tri thức hệ thống.....	28
3.6.	Thiết kế cơ sở dữ liệu	29
3.6.1.	Lược đồ ảo trực quan hóa cơ sở dữ liệu	29
3.6.2.	Thiết kế chi tiết các bảng	30
3.7.	Thiết kế hệ thống	31
3.7.1.	Tổng quan kiến trúc AI Agents	31

3.7.2.	Phân tích chi tiết các thành phần trong hệ thống tác tử	32
3.7.3.	Thiết kế Prompt (Prompt Engineering)	35
3.7.4.	Kết quả thực hiện	37
3.8.	Thiết kế giao diện người dùng (User Interface – UI)	40
3.8.1.	Giao diện trang chat chính	40
3.8.2.	Giao diện trang xác thực đăng nhập	41
3.8.3.	Giao diện trang xác thực đăng ký	41
3.8.4.	Giao diện khi thực hiện các chức năng chính của hệ thống	42
Chương 4.	KẾT LUẬN VÀ XÂY DỰNG HƯỚNG PHÁT TRIỂN	44
4.1.	Kết luận	44
4.2.	Những điều đã đạt được và điều chưa thực hiện	44
4.2.1.	Những điều đạt được	44
4.2.2.	Những hạn chế và hướng phát triển	44
TÀI LIỆU THAM KHẢO		46

DANH MỤC TỪ VIẾT TẮT

STT	Từ viết tắt	Từ đầy đủ	Ý nghĩa
1	AI	Artificial Intelligence	Trí tuệ nhân tạo
2	LLM	Large Language Model	Mô hình Ngôn ngữ lớn
3	RAG	Retrieval Augmented Generation	Tạo tăng cường truy xuất
4	ReAct	Reason + Act	Suy luận + Hành động
5	API	Application Programming Interface	Giao diện lập trình ứng dụng
6	UI	User Interface	Giao diện người dùng
7	ORM	Object-Relational Mapping	Ánh xạ Đối tượng – Quan hệ
8	OCR	Optical Character Recognition	Nhận diện ký tự quang học
9	PDF	Portable Document Format	Định dạng tài liệu di động
10	FAISS	Facebook AI Similarity Search	Thư viện tìm kiếm tương đồng của Facebook
11	NLP	Natural Language Processing	Xử lý Ngôn ngữ tự nhiên
12	JSON	JavaScript Object Notation	Ký pháp đối tượng JavaScript

DANH MỤC HÌNH ẢNH

Hình 1: Trí tuệ nhân tạo tạo sinh – Generative AI	4
Hình 2: Cơ chế hoạt động của Generative AI	5
Hình 3: Mô hình Ngôn ngữ lớn – Large Language Model (LLM)	5
Hình 4: Tổng quan quá trình huấn luyện.....	6
Hình 5: Hallucination trong LLM là gì?.....	7
Hình 6: Tổng quan về Vector Embedding.....	8
Hình 7: Quá trình làm việc với một Vector Store	9
Hình 8: Retrieval-Augmented Generation (RAG) là gì?.....	10
Hình 9: Giai đoạn Lập chỉ mục (Indexing)	11
Hình 10: Giai đoạn Truy xuất (Retireval)	11
Hình 11: Tổng quan quá trình của RAG	12
Hình 12: Tác tử AI (AI Agent) là gì?	13
Hình 13: Tác tử Simple Reflex Agents	14
Hình 14: Tác tử Model-based Reflex Agents.....	14
Hình 15: Tác tử Goal-based Agents	15
Hình 16: Tác tử Utility-based Agents	15
Hình 17: LangChain	16
Hình 18: Cách thức hoạt động của LangChain	17
Hình 19: Tổng quan kiến trúc hệ thống.....	19
Hình 20: Lược đồ use case của hệ thống.....	22
Hình 21: Lược đồ ảo trực quan hóa CSDL	29
Hình 22: Luồng xử lý từ khi nhận câu hỏi đến khi trả lời.....	32
Hình 23: Mã nguồn cho Rewriter Chain	33
Hình 24: Mã nguồn cho Router Chain	34
Hình 25: Mã nguồn cho ReAct Agent.....	34
Hình 26: Mã nguồn cho Conversational Chain	35
Hình 27: Mã nguồn prompt cho Tác tử ReAct.....	36
Hình 28: Giao diện chính của hệ thống.....	40
Hình 29: Giao diện trang đăng nhập của hệ thống	41
Hình 30: Giao diện trang đăng ký tài khoản	41
Hình 31: Giao diện hệ thống khi người dùng đăng nhập thành công.....	42

Hình 32: Giao diện hệ thống khi người dùng đăng nhập thành công.....	42
Hình 33: Giao diện hỏi – đáp của hệ thống.....	43
Hình 34: Giao diện khi Người dùng thực hiện tìm kiếm hội thoại	43

DANH MỤC BẢNG

Bảng 1: Bảng đặc tả chức năng use case đăng ký người dùng.....	23
Bảng 2: Bảng đặc tả chức năng use case đăng nhập	24
Bảng 3: Bảng đặc tả chức năng use case đưa ra câu hỏi	24
Bảng 4: Bảng đặc tả chức năng use case trả lời câu hỏi.....	25
Bảng 5: Bảng đặc tả chức năng use case tiếp tục ngữ cảnh câu hỏi.....	26
Bảng 6: Bảng đặc tả chức năng use case tìm kiếm đoạn hội thoại.....	27
Bảng 7: Bảng đặc tả chức năng use case xóa hội thoại	28
Bảng 8: Bảng đặc tả chức năng use cập nhật cơ sở tri thức hệ thống	29
Bảng 9: Chi tiết bảng users.....	30
Bảng 10: Chi tiết bảng chat_sessions	30
Bảng 11: Chi tiết bảng messages	31
Bảng 12: Phân tích chi tiết các thành phần trong prompt	37

MỞ ĐẦU

Trong bối cảnh cuộc Cách mạng Công nghiệp 4.0 đang có tác động sâu rộng đến mọi lĩnh vực, ngành Nông nghiệp Việt Nam cũng đang có những bước chuyển mình mạnh mẽ theo hướng hiện đại hóa. Người nông dân ngày càng chủ động hơn trong việc tìm kiếm và tiếp cận kiến thức kỹ thuật thông qua các kênh thông tin đa dạng như website, diễn đàn, và các hội nhóm trên mạng xã hội. Đây là những công cụ hữu ích, giúp lan tỏa tri thức và kết nối cộng đồng sản xuất một cách nhanh chóng.

Tuy nhiên, song song với những cơ hội đó là một thách thức lớn: *sự nhiễu loạn thông tin*. Lượng kiến thức nông nghiệp trên không gian mạng tuy khổng lồ nhưng lại bị phân mảnh, thiếu sự kiểm chứng và thường không nhất quán. Người nông dân dễ bị lạc vào một “ma trận” thông tin với những bài viết chung chung, các hướng dẫn lỗi thời, hoặc những kinh nghiệm không phù hợp với điều kiện thổ nhưỡng, khí hậu tại địa phương của họ.

Việc tiếp nhận và áp dụng thông tin kỹ thuật thiếu chính xác có thể gây ra những hậu quả nghiêm trọng và trực tiếp. Thực tế đã cho thấy nhiều trường hợp nông dân chịu thiệt hại kinh tế nặng nề do áp dụng các quy trình canh tác không phù hợp được chia sẻ tràn lan. Một ví dụ điển hình là các chiến dịch “*Giải cứu nông sản*” diễn ra gần như hằng năm. Tình trạng này bắt nguồn từ việc nông dân ồ ạt trồng một loại cây theo trào lưu được lan truyền trên mạng xã hội mà thiếu đi sự phân tích thị trường kỹ lưỡng, dẫn đến khủng hoảng thừa, giá cả lao dốc và thua lỗ nặng nề. Những sai lầm trong việc lựa chọn cây trồng dựa trên thông tin không đáng tin cậy này không chỉ gây lãng phí nguồn lực, làm giảm giá trị nông sản mà còn có thể tác động tiêu cực đến quy hoạch sản xuất chung.

Xuất phát từ thực tiễn trên, đề tài “Ứng dụng Retrieval-Augmented Generation (RAG) xây dựng hệ thống tác tử AI trong nông nghiệp thông minh” được thực hiện. Mục tiêu của đề tài là xây dựng một công cụ thông minh, đóng vai trò như một “chuyên gia số” đáng tin cậy. Hệ thống này sẽ cung cấp cho người nông dân một kênh truy vấn tập trung, nơi họ có thể nhận được những thông tin kỹ thuật đã được kiểm chứng và các khuyến nghị phù hợp, từ đó giúp họ đưa ra quyết định canh tác hiệu quả hơn và giảm thiểu rủi ro trong sản xuất.

Chương 1. GIỚI THIỆU ĐỀ TÀI

1.1. Lý do chọn đề tài

Trong **Quyết định số 749/QĐ-TTg** phê duyệt “Chương trình Chuyển đổi số quốc gia đến năm 2025, định hướng đến năm 2030”, Nông nghiệp được xác định là một trong tám lĩnh vực cần ưu tiên chuyển đổi số hàng đầu. Điều này cho thấy tầm nhìn và quyết tâm của Chính phủ trong việc hiện đại hóa ngành nông nghiệp. Tuy nhiên, để quá trình chuyển đổi số này thành công và đi vào thực tế, một trong những rào cản ở cấp độ người sản xuất cần được giải quyết chính là vấn đề truy cập thông tin kỹ thuật đáng tin cậy. Hiện tại, người nông dân đang phải đối mặt với tình trạng nhiễu loạn thông tin từ nhiều nguồn không được kiểm chứng, gây ra khó khăn cho việc ra quyết định và tiềm ẩn nhiều rủi ro.

Để góp phần giải quyết phần nào rào cản đó, đề tài lựa chọn một hướng tiếp cận công nghệ tiên tiến là xây dựng **một tác tử AI** ứng dụng kiến trúc RAG. Khác với các AI thông thường có nguy cơ tự “bịa” ra thông tin, kỹ thuật RAG đảm bảo mọi câu trả lời đều được truy xuất và tổng hợp từ một cơ sở tri thức gồm các tài liệu nông nghiệp uy tín.

1.2. Mục tiêu nghiên cứu

Mục tiêu chính của đề tài là xây dựng được một hệ thống **tác tử AI tên AgriBot**, ứng dụng kiến trúc RAG để cung cấp một kênh thông tin kỹ thuật nông nghiệp đáng tin cậy, hỗ trợ người nông dân ra quyết định canh tác hiệu quả. Để thực hiện điều này, quá trình nghiên cứu sẽ bắt đầu bằng việc hệ thống hóa cơ sở lý thuyết về Mô hình Ngôn ngữ lớn và kiến trúc RAG, tiếp đó là **xây dựng một cơ sở tri thức chuyên ngành** từ các tài liệu nông nghiệp uy tín. Trên nền tảng đó, lõi tác tử AI sẽ được phát triển, tích hợp thành công kiến trúc RAG và các công cụ hỗ trợ như API thời tiết để mở rộng năng lực phân tích. Song song, một **giao diện ứng dụng website thân thiện** sẽ được thiết kế nhằm đảm bảo người dùng có thể tương tác với hệ thống một cách dễ dàng.

1.3. Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài bao gồm:

Dữ liệu tài liệu nông nghiệp: Nghiên cứu các tài liệu kỹ thuật, giáo trình nông nghiệp được lưu dưới dạng PDF.

Kiến trúc Tác tử AI và RAG: Tập trung vào mô hình tác tử ReAct và kiến trúc RAG để truy xuất thông tin, suy luận và tạo sinh câu trả lời chính xác từ cơ sở tri thức đã xây dựng.

Hệ thống ứng dụng web: Nghiên cứu việc triển khai toàn bộ hệ thống trên nền tảng web, bao gồm Flask, JavaScript và cơ sở dữ liệu MySQL để quản lý người dùng và phiên trò chuyện.

1.4. Bố cục đề tài

Chương 1: Giới thiệu đề tài

Chương 2: Các công nghệ và cơ sở lý thuyết

Chương 3: Xây dựng hệ thống tác tử AI trong nông nghiệp thông minh ứng dụng RAG

Chương 4: Kết luận và phương hướng phát triển của hệ thống trong tương lai

Chương 2. CÁC CÔNG NGHỆ VÀ CƠ SỞ LÝ THUYẾT

2.1. Trí tuệ nhân tạo tạo sinh và mô hình Ngôn ngữ lớn (LLM)

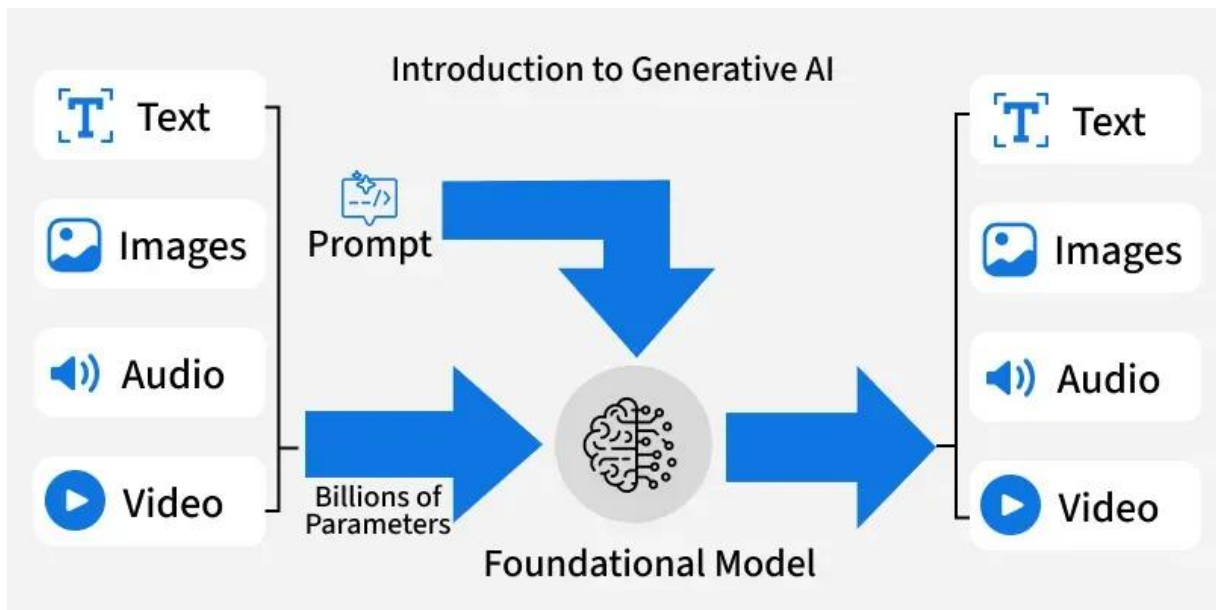
2.1.1. Trí tuệ nhân tạo tạo sinh (Generative AI)

Trí tuệ nhân tạo tạo sinh (Generative AI) là một nhánh của Trí tuệ nhân tạo tập trung vào việc **tạo ra nội dung mới và nguyên bản** dưới nhiều hình thức khác nhau, bao gồm văn bản, hình ảnh, âm nhạc, và mã nguồn. Thay vì chỉ thực hiện các tác vụ phân tích hay phân loại dữ liệu có sẵn như các mô hình AI truyền thống (còn gọi là AI phân biệt – Discriminative AI), các mô hình AI tạo sinh có khả năng tự sản sinh ra dữ liệu mới.



Hình 1: Trí tuệ nhân tạo tạo sinh – Generative AI

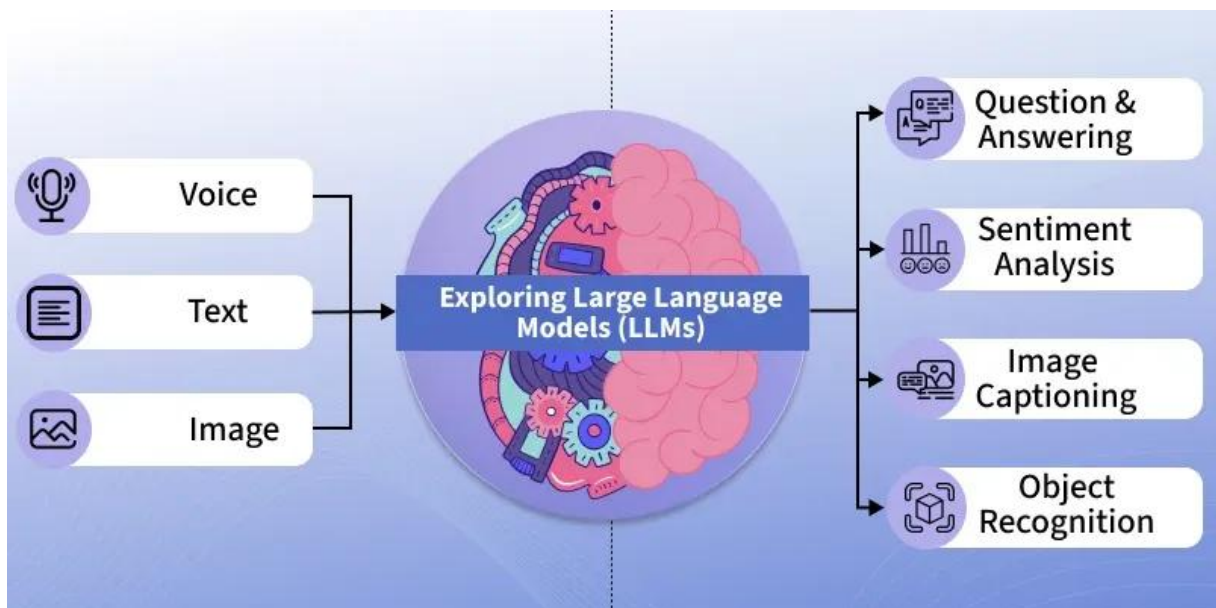
Cơ chế hoạt động của AI tạo sinh dựa trên việc học hỏi các mẫu (patterns) và cấu trúc tiềm ẩn từ một tập dữ liệu huấn luyện khổng lồ. Sau khi học được những đặc trưng này, mô hình có thể sử dụng kiến thức đó để tạo ra các sản phẩm mới có những đặc điểm tương tự như dữ liệu gốc nhưng hoàn toàn độc đáo. Quá trình này có thể được hình dung như một nghệ sĩ học hỏi phong cách của nhiều họa sĩ khác nhau để rồi tự mình sáng tác ra một bức tranh mới mang phong cách riêng.



Hình 2: Cơ chế hoạt động của Generative AI

2.1.2. Mô hình Ngôn ngữ lớn (Large Language Model – LLM)

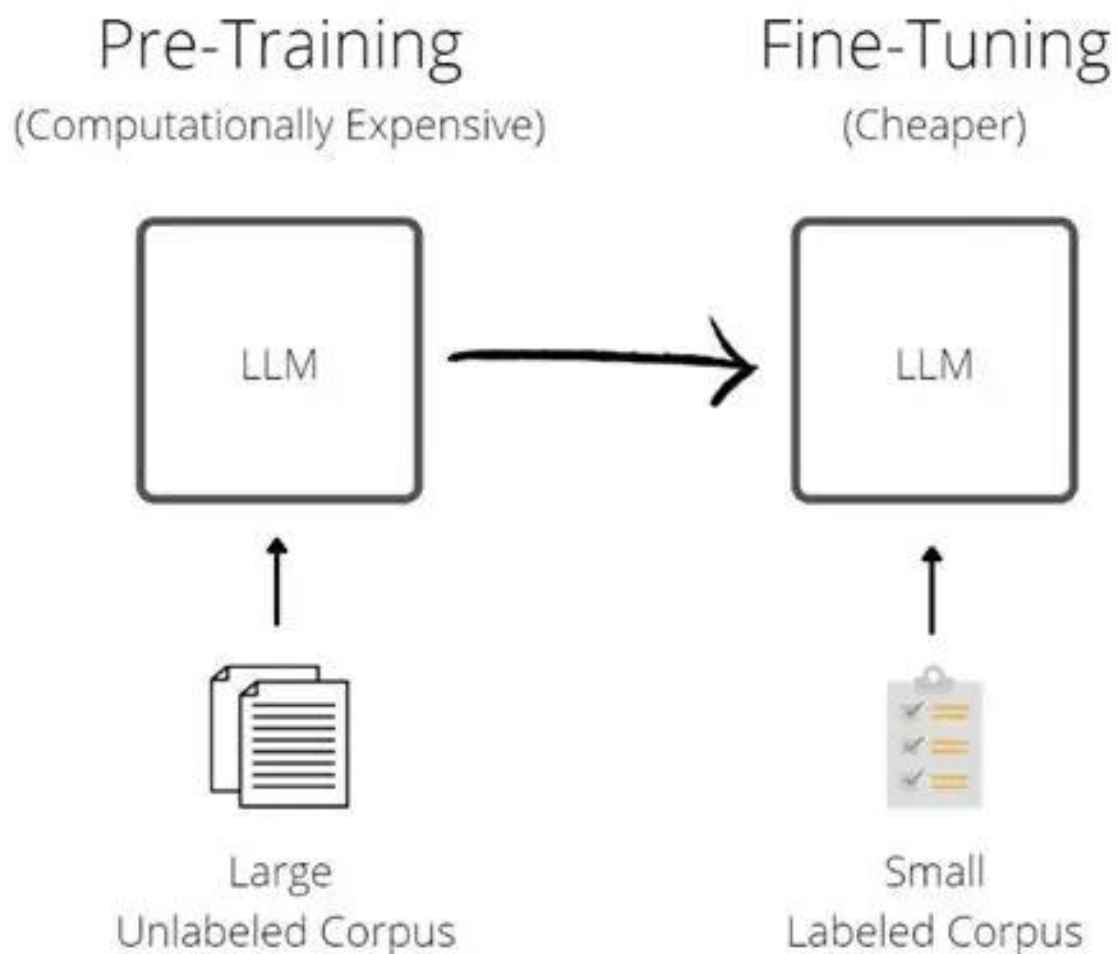
Mô hình Ngôn ngữ lớn (LLM) là một loại mô hình Trí tuệ nhân tạo (AI) được huấn luyện với dữ liệu văn bản khổng lồ để hiểu và tạo ra ngôn ngữ của con người một cách tinh vi. Chúng là các mạng nơ-ron sâu, thường có hàng tỷ tham số, cho phép chúng nắm bắt các mẫu, ngữ pháp, ngữ nghĩa phức tạp và thậm chí cả kiến thức thực tế có trong dữ liệu huấn luyện. Về bản chất, LLM là các mô hình nền tảng (Foundation Models) có thể được điều chỉnh để thực hiện nhiều tác vụ ngôn ngữ tự nhiên khác nhau.



Hình 3: Mô hình Ngôn ngữ lớn – Large Language Model (LLM)

Nền tảng của hầu hết các LLM hiện đại là kiến trúc **Transformer**, được giới thiệu lần đầu vào năm 2017. Kiến trúc này xử lý dữ liệu văn bản dưới dạng các chuỗi và sử dụng một cơ chế gọi là **tự chú ý (self-attention)**. Cơ chế này cho phép mô hình cân nhắc tầm quan trọng của các từ khác nhau trong một câu khi xử lý và tạo ra văn bản, giúp nó hiểu được ngữ cảnh và các mối quan hệ phức tạp giữa các từ.

Quá trình huấn luyện của một LLM thường diễn ra theo hai giai đoạn: **Tiền huấn luyện (Pre-training)** và **Tinh chỉnh (Fine-tuning)**.



Hình 4: Tổng quan quá trình huấn luyện

2.1.3. Vấn đề “Ảo giác” (Hallucination) trong LLM

Mặc dù sở hữu năng lực ngôn ngữ vượt trội, các Mô hình Ngôn ngữ lớn (LLM) tồn tại một điểm yếu cố hữu và rất nguy hiểm được gọi là “**ảo giác**” (**hallucination**). Đây là hiện tượng mô hình tạo ra các thông tin sai lệch, vô nghĩa, không chính xác về mặt sự

thật, hoặc không liên quan đến ngữ cảnh được cung cấp, nhưng lại trình bày chúng một cách rất tự tin và thuyết phục.



Hình 5: Hallucination trong LLM là gì? 

Về bản chất, LLM không “nói dối” một cách có chủ ý. Thay vào đó, nó hoạt động giống như một “cỗ máy mơ mộng”, liên tục trộn lẫn và kết hợp lại các mẫu thông tin mà nó đã học được từ dữ liệu huấn luyện khổng lồ. Mô hình không sở hữu một hệ thống nội tại để kiểm chứng sự thật. Mục tiêu chính của nó là dự đoán từ tiếp theo có khả năng xuất hiện cao nhất để tạo ra một chuỗi văn bản mạch lạc và hợp lý, chứ không phải từ chính xác nhất về mặt sự thật.

Hiện tượng “ảo giác” này xảy ra chủ yếu do hai nguyên nhân chính sau:

Lỗi hổng và thiên kiến trong dữ liệu huấn luyện: Dữ liệu huấn luyện của LLM, dù rất lớn, không thể bao quát toàn bộ kiến thức của nhân loại và có thể chứa đựng các thông tin sai lệch hoặc thiên kiến. Khi gặp phải một chủ đề mà nó có ít thông tin, mô hình có xu hướng “lấp đầy khoảng trống” bằng cách bịa ra các chi tiết có vẻ hợp lý.

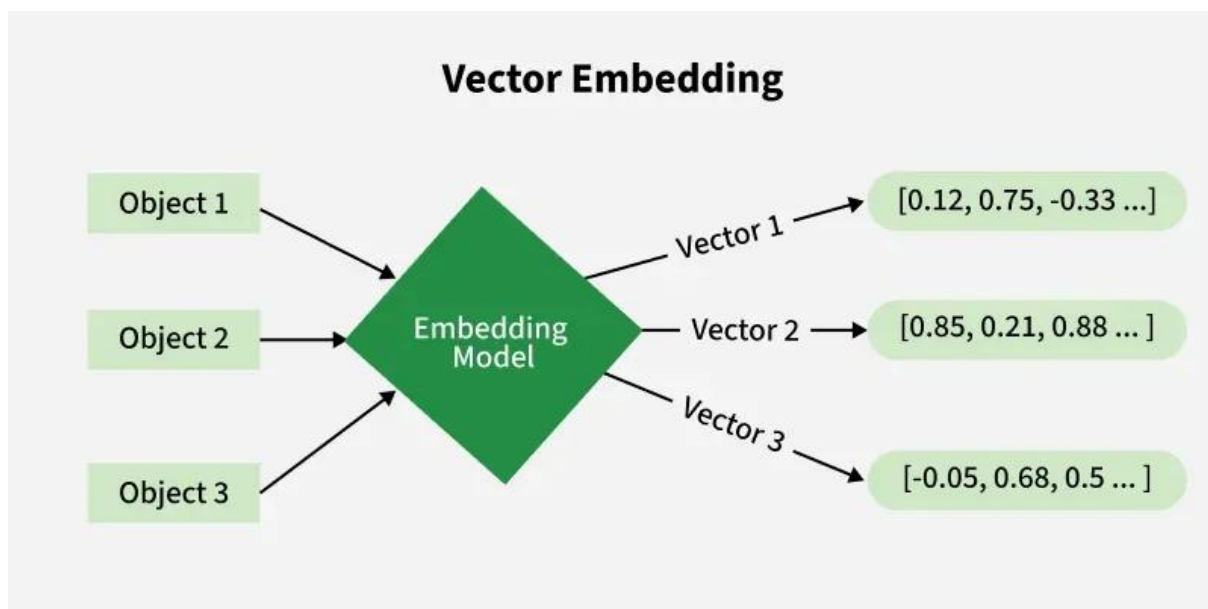
Bản chất Xác suất của Mô hình: LLM hoạt động dựa trên việc dự đoán từ tiếp theo. Đôi khi, một thông tin sai lệch lại là lựa chọn có xác suất cao nhất để tạo ra một câu văn nghe có vẻ tự nhiên và trôi chảy.

2.2. Vector Embedding và Cơ sở dữ liệu Vector (Vector Store)

Để máy tính có thể “hiểu” và tìm kiếm văn bản dựa trên ngữ nghĩa thay vì từ khóa, chúng ta cần một phương pháp để biểu diễn ý nghĩa của văn bản dưới dạng số học. Vector Embedding chính là giải pháp cho bài toán này.

2.2.1. Vector Embedding

Vector Embedding là một kỹ thuật trong Xử lý Ngôn ngữ tự nhiên (NLP) dùng được để biểu diễn các từ, câu, hoặc toàn bộ tài liệu dưới dạng các vector số học trong một không gian nhiều chiều. Không giống như các phương pháp truyền thống như one-hot coding tạo ra các vector thưa thớt (sparse) và không thể hiện được mối quan hệ ngữ nghĩa, vector embedding tạo ra các vector dày đặc (dense), chứa đựng các thông tin về ngữ cảnh và ý nghĩa của văn bản.

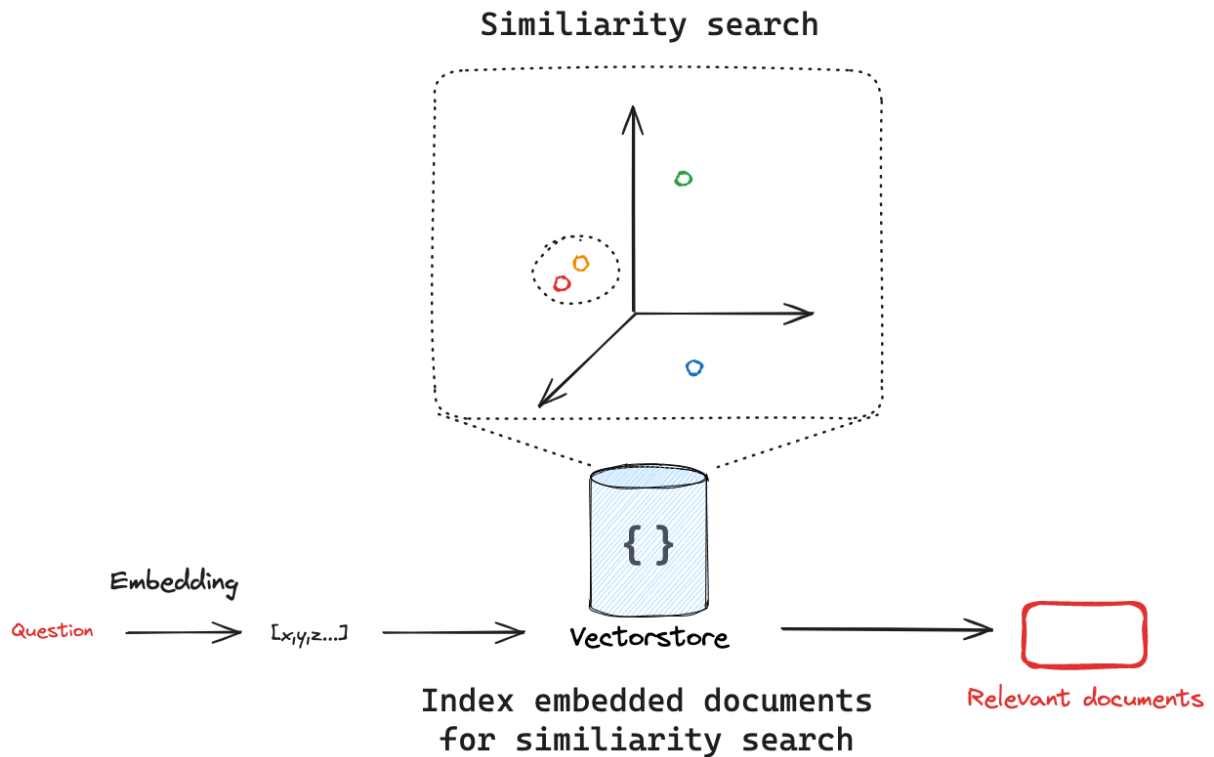


Hình 6: Tổng quan về Vector Embedding

Tầm quan trọng của Vector Embedding nằm ở khả năng nắm bắt được sự tương đồng về mặt ngữ nghĩa. Các mô hình như Word2Vec, GloVe, và các mô hình dựa trên Transformer (như BERT) được huấn luyện trên các kho văn bản khổng lồ để học cách biểu diễn này. Nguyên tắc cốt lõi là các từ hoặc câu xuất hiện trong những ngữ cảnh tương tự sẽ có các vector biểu diễn nằm gần nhau trong không gian vector. Nhờ đó, các mô hình học máy có thể thực hiện các tác vụ phức tạp bằng cách so sánh sự gần gũi giữa các vector này, cho phép chúng hiểu được các mối quan hệ như từ đồng nghĩa hoặc các chủ đề liên quan.

2.2.2. Cơ sở dữ liệu Vector (Vector Store)

Vector Store là một hệ thống được thiết kế chuyên biệt để lưu trữ các văn bản phi cấu trúc và các vector embedding tương ứng của chúng. Chức năng chính và quan trọng nhất của một Vector Store là cung cấp khả năng tìm kiếm các đoạn văn bản có ngữ nghĩa tương đồng nhất với một câu hỏi hoặc một đoạn văn bản cho trước.



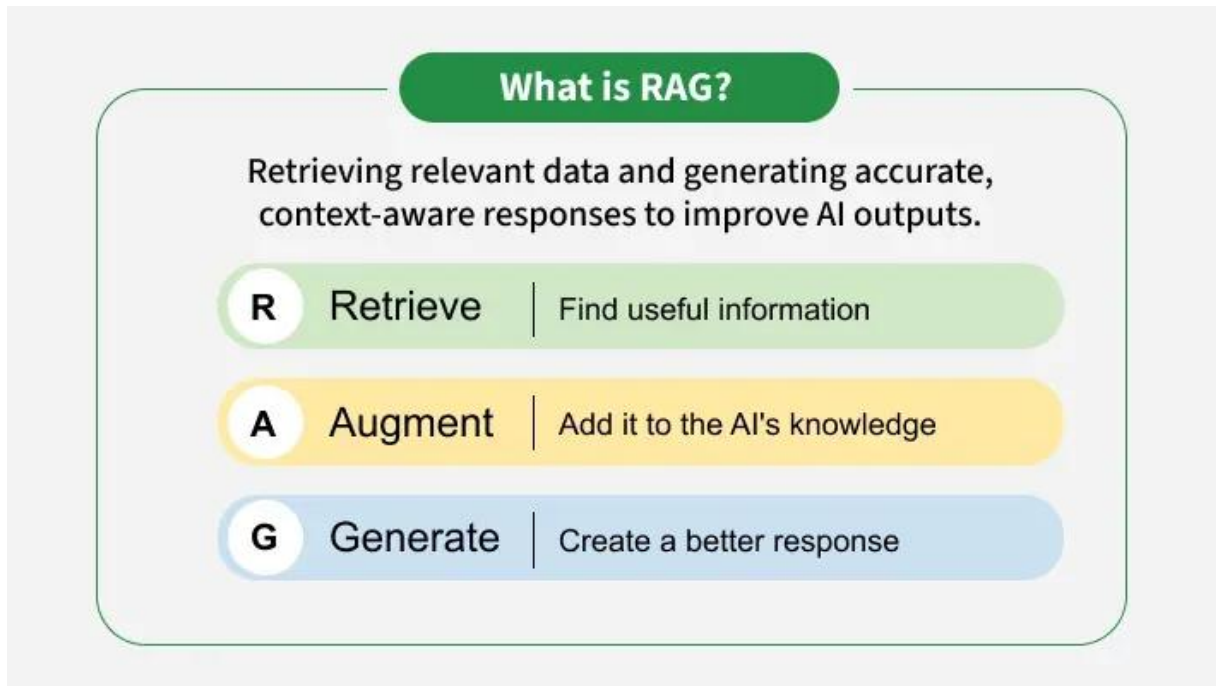
Hình 7: Quá trình làm việc với một Vector Store

Sau khi dữ liệu đã được lưu trữ, người dùng có thể thực hiện truy vấn. Vector Store sẽ nhận một câu hỏi, chuyển nó thành một vector, và sau đó thực hiện **tìm kiếm tương đồng (similarity search)** để tìm ra các vector (và các đoạn văn bản tương ứng) gần nhất với vector của câu hỏi trong cơ sở dữ liệu. Các đoạn văn bản này sau đó sẽ được sử dụng làm ngữ cảnh cho Mô hình Ngôn ngữ lớn, là bước nền tảng cho kiến trúc RAG.

2.3. Kiến trúc Retrieval-Augmented Generation (RAG)

Đây là một kiến trúc được thiết kế để nâng cao hiệu suất và giải quyết vấn đề “ảo giác” (hallucination) của LLM. **Retrieval-Augmented Generation (RAG)** là một phương pháp kết hợp sức mạnh của các hệ thống truy xuất thông tin với khả năng tạo sinh ngôn ngữ của các LLM. Thay vì chỉ dựa vào kiến thức tĩnh đã được huấn luyện, RAG cho

phép mô hình truy cập và tham chiếu đến một **cơ sở tri thức có thẩm quyền bên ngoài** trước khi tạo ra câu trả lời.

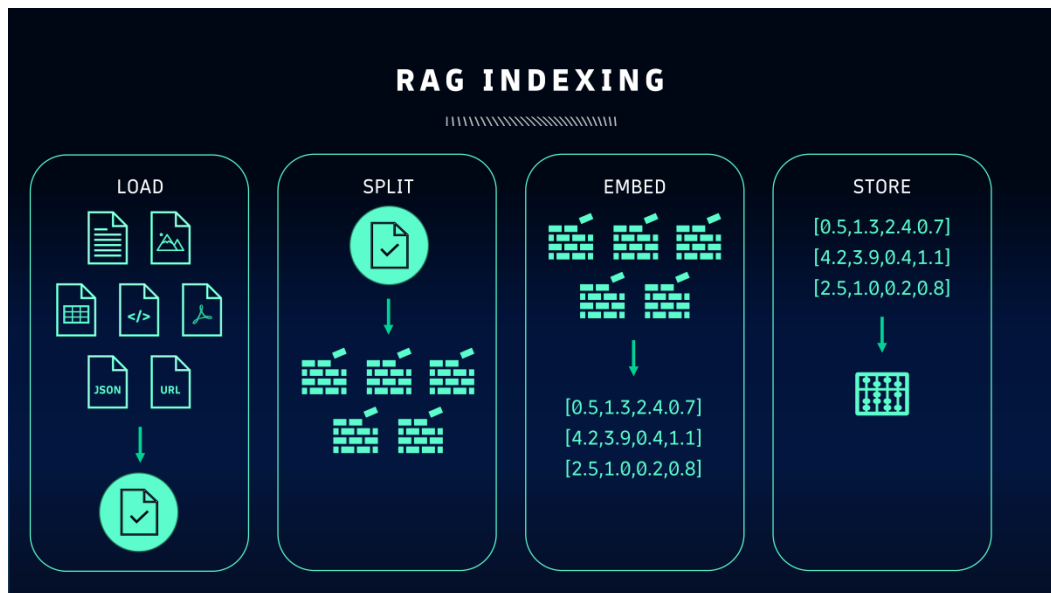


Hình 8: Retrieval-Augmented Generation (RAG) là gì?

Kiến trúc RAG hoạt động theo một quy trình ba bước chính: **Lập chỉ mục (Indexing)**, **Truy xuất (Retrieval)** và **Tạo sinh (Generation)**.

2.3.1. Giai đoạn 1: Lập chỉ mục (Indexing)

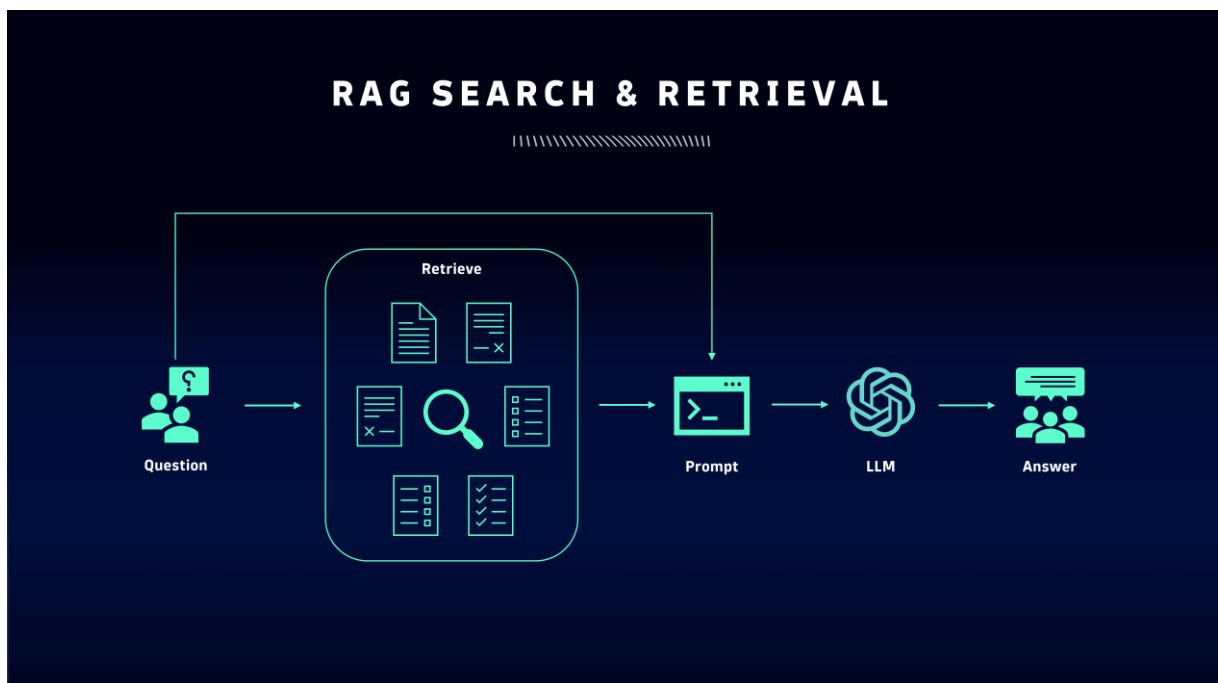
Đây là một quy trình nền tảng để chuẩn bị dữ liệu. Các tài liệu từ các nguồn bên ngoài được tải, làm sạch, và chia thành các đoạn văn bản nhỏ hơn (chunks). Sau đó, mỗi chunk được chuyển đổi thành vector embedding và được lưu trữ trong một cơ sở dữ liệu vector. Quá trình này tạo ra một thư viện kiến thức mà mô hình có thể tìm kiếm được.



Hình 9: Giai đoạn Lập chỉ mục (Indexing)

2.3.2. Giai đoạn 2: Truy xuất (Retrieval)

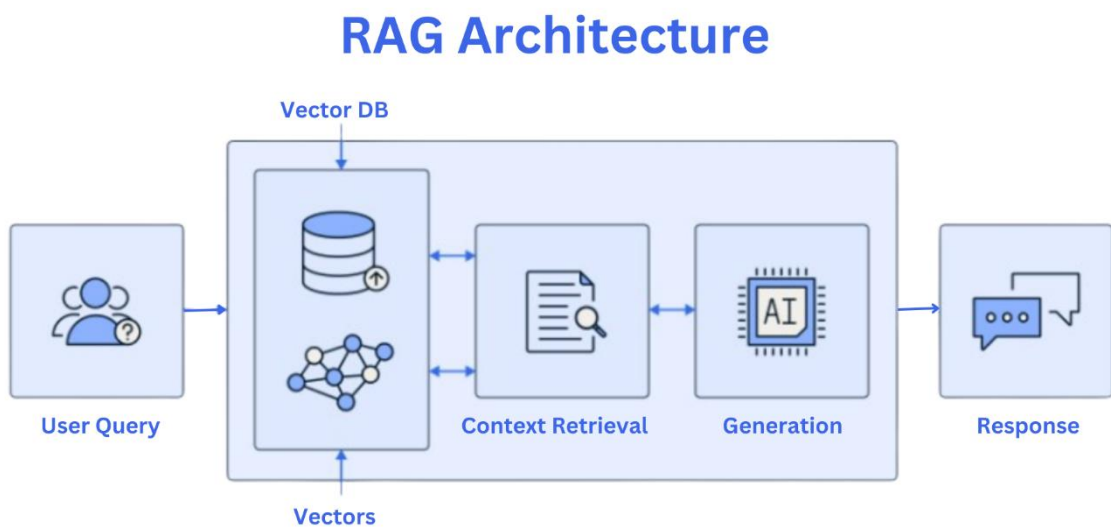
Đây là bước đi tìm kiếm thông tin khi có yêu cầu từ người dùng. Câu hỏi của người dùng được chuyển đổi thành một vector. Vector này sau đó được sử dụng để tìm kiếm trong cơ sở dữ liệu vector nhằm xác định và lấy ra các đoạn văn bản (chunks) có nội dung tương đồng về mặt ngữ nghĩa nhất.



Hình 10: Giai đoạn Truy xuất (Retireval)

2.3.3. Giai đoạn 3: Tạo sinh tăng cường (Augmented Generation)

Sau khi có được một tập hợp các thông tin liên quan từ giai đoạn truy xuất, hệ thống sẽ thực hiện bước cuối cùng. Một **prompt (lời nhắc)** mới sẽ được xây dựng, bao gồm câu hỏi gốc của người dùng và toàn bộ các thông tin đã được truy xuất để làm **ngữ cảnh (context)**. Prompt hoàn chỉnh này sau đó mới được gửi đến mô hình Ngôn ngữ lớn. LLM được chỉ thị rõ ràng rằng nó phải trả lời câu hỏi **chỉ dựa trên ngữ cảnh được cung cấp**, qua đó đảm bảo câu trả lời vừa tự nhiên, mạch lạc, vừa bám sát sự thật từ cơ sở tri thức.



Hình 11: Tổng quan quá trình của RAG

2.4. Tác tử AI (AI Agent)

2.4.1. Khái niệm Tác tử AI

Trong lĩnh vực Trí tuệ nhân tạo, một **tác tử (agent)** được định nghĩa là bất kỳ thực thể nào có khả năng **cảm nhận môi trường** của nó thông qua các **cảm biến (sensors)** và **hành động lên môi trường** đó thông qua các **cơ cấu truyền động (actuators)**. Ví dụ, cảm biến của một robot tự hành có thể là camera và cảm biến hồng ngoại, trong khi cơ cấu truyền động của nó là động cơ và bánh xe.



Hình 12: Tác tử AI (AI Agent) là gì?

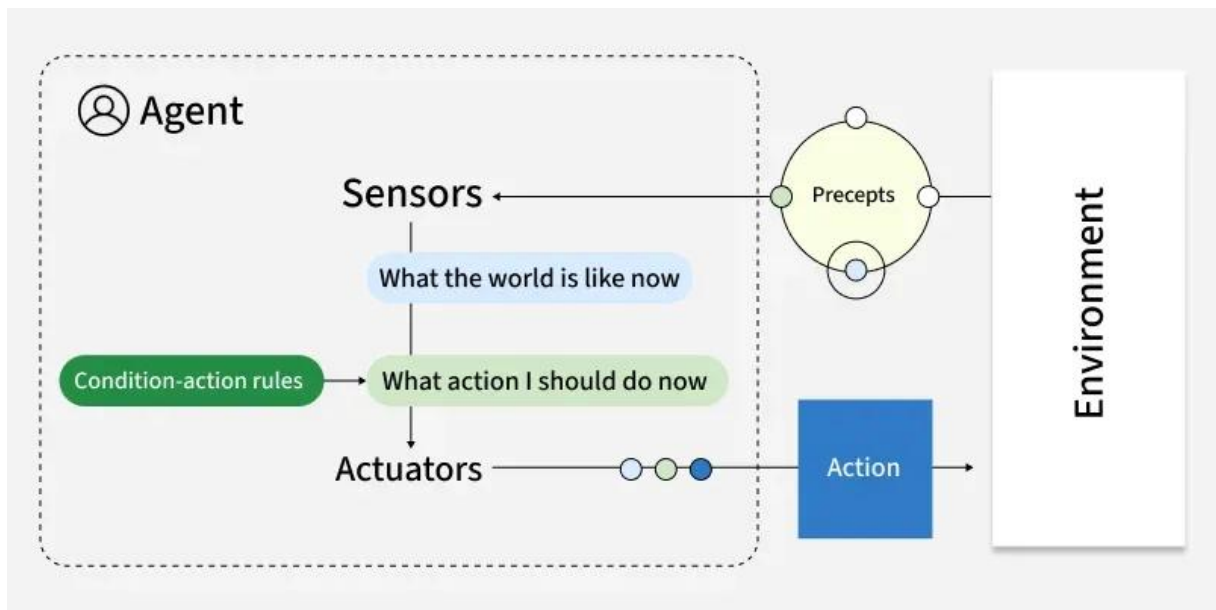


Một tác tử AI được coi là một **tác tử hợp lý (rational agent)** nếu nó luôn thực hiện hành động được cho là sẽ mang lại kết quả tốt nhất để tối đa hóa thước đo hiệu suất của nó. Để mô tả và thiết kế một tác tử hợp lý, người ta thường sử dụng mô hình **PEAS (Performance Measure, Environment, Actuators, Sensors)** để xác định các thành phần của bài toán toán mà tác tử cần giải quyết.

2.4.2. Các loại Tác tử AI

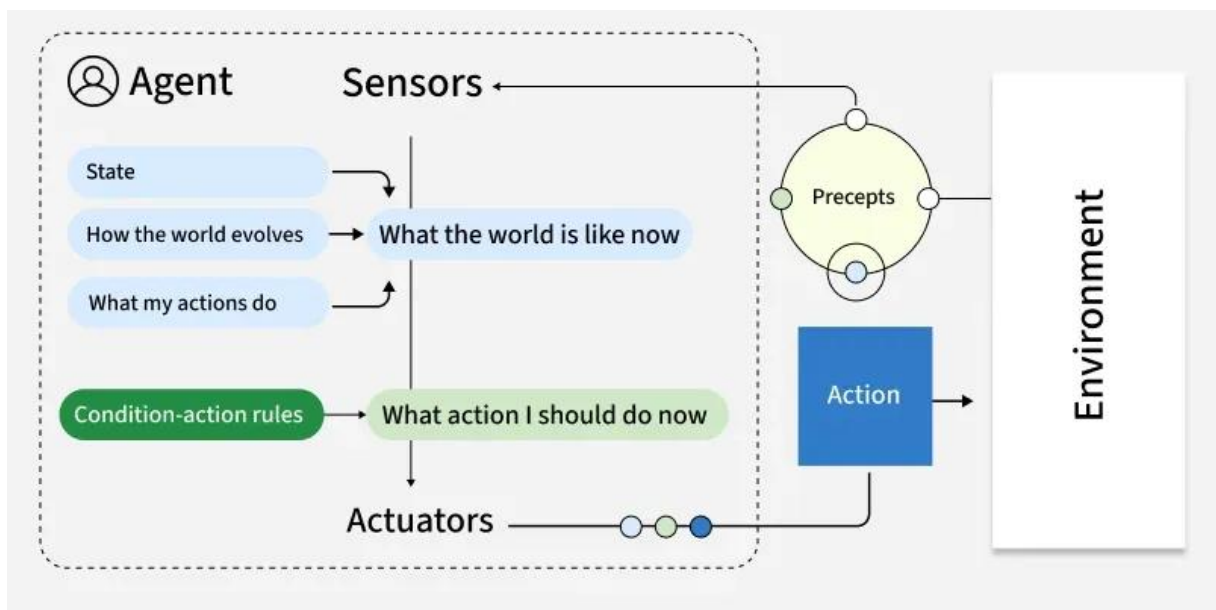
Dựa trên mức độ thông minh và khả năng nhận thức, các tác tử AI có thể được phân loại khác nhau. Một số tác tử phổ biến như:

Tác tử phản xạ đơn giản (Simple Reflex Agents): Các tác tử này lựa chọn hành động chỉ dựa trên những gì cảm nhận được ở hiện tại (current percept), bỏ qua lịch sử các cảm nhận trước đó. Chúng hoạt động dựa trên các quy tắc điều kiện – hành động (condition – action rules).



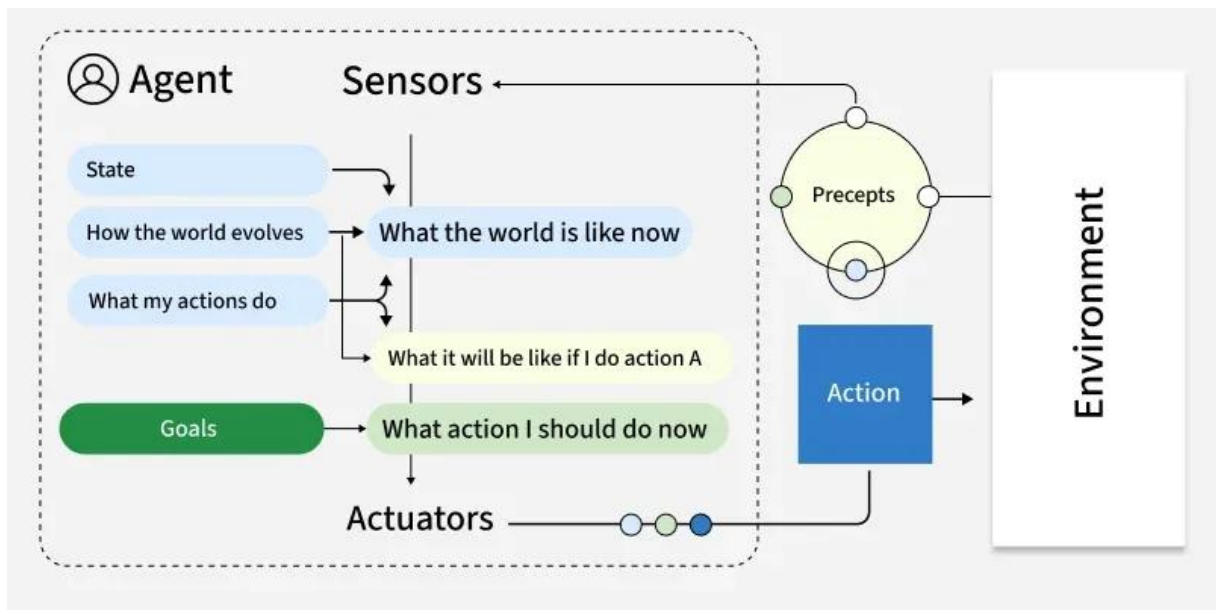
Hình 13: Tác tử Simple Reflex Agents

Tác tử dựa trên mô hình (Model-based Reflex Agents): Các tác tử này duy trì một trạng thái nội tại (internal state) để theo dõi các khía cạnh của môi trường mà cảm biến hiện tại không thể thấy được. Trạng thái này được cập nhật dựa trên lịch sử cảm nhận và các hành động đã được thực hiện.



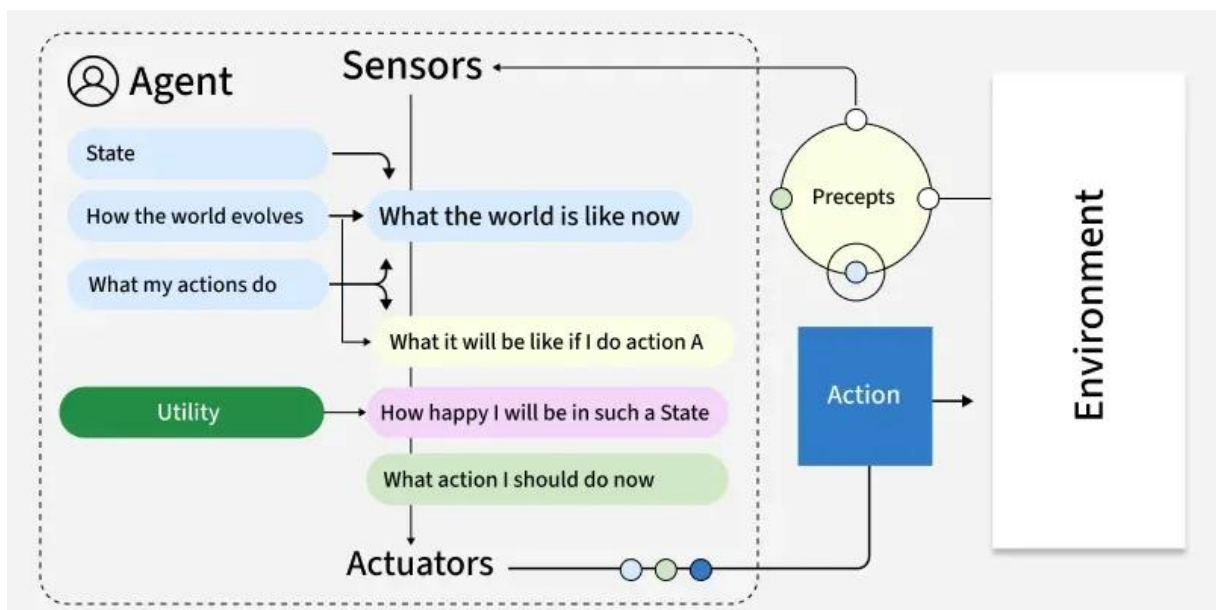
Hình 14: Tác tử Model-based Reflex Agents

Tác tử dựa trên mục tiêu (Goal-based Agents): Ngoài trạng thái nội tại, các tác tử này còn có thông tin về “mục tiêu” (goal) cần đạt được. Chúng lựa chọn hành động để tiến gần hơn đến việc hoàn thành mục tiêu đó, cho phép chúng có khả năng lập kế hoạch và ra quyết định linh hoạt hơn.



Hình 15: Tác tử Goal-based Agents

Tác tử dựa trên sự hữu dụng (Utility-based Agents): Đây là một dạng nâng cao của tác tử dựa trên mục tiêu. Khi có nhiều cách để đạt được mục tiêu, tác tử này sẽ lựa chọn cách mang lại “sự hữu dụng” (utility) cao nhất, tức là trạng thái được ưu tiên nhất hoặc “hạnh phúc nhất”.



Hình 16: Tác tử Utility-based Agents

2.5. LangChain

2.5.1. **Khai** niệm LangChain



LangChain là một framework mã nguồn mở được thiết kế đơn giản hóa và tăng tốc quá trình phát triển các ứng dụng dựa trên Mô hình Ngôn ngữ lớn (LLM). Nó cung cấp các dụng cụ cùng chung hệ sinh thái để xây dựng các phần mềm, triển khai và quản lý hiệu suất, mở rộng hệ thống.



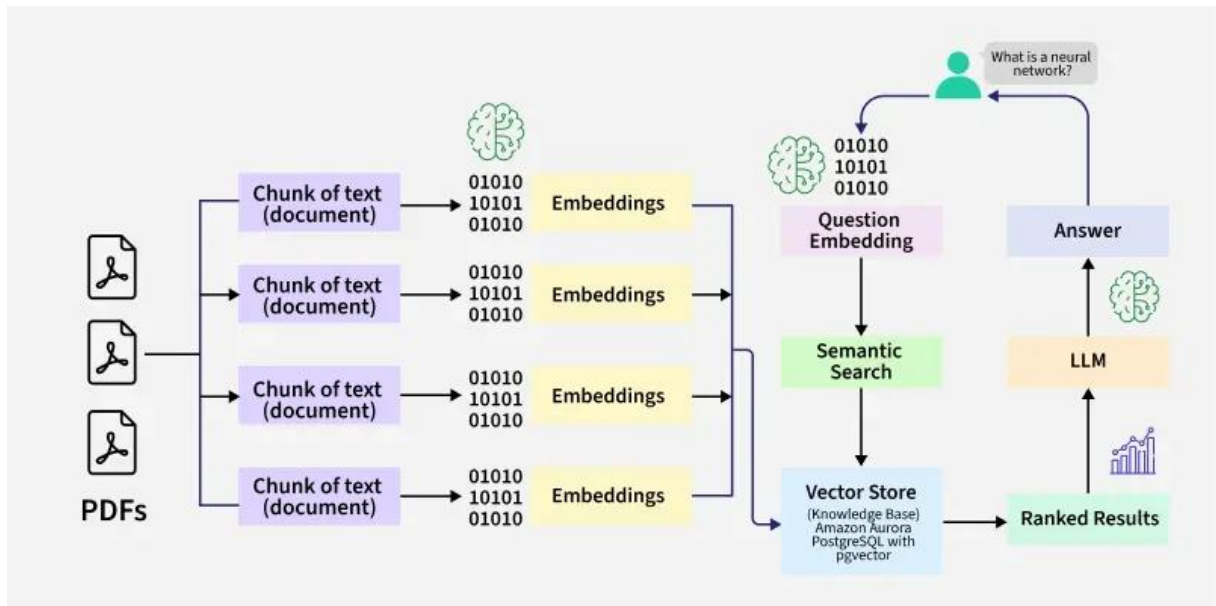
Hình 17: LangChain

2.5.2. Nguyên lý hoạt động và mục tiêu

Về cơ bản, một LLM hoạt động như một bộ não bị cô lập, không có nhận thức về thế giới bên ngoài ngoài những kiến thức tĩnh đã được huấn luyện. Mục tiêu của LangChain là phá vỡ sự cô lập này bằng cách cung cấp một bộ khung tiêu chuẩn để kết nối LLM với các thành phần khác, giúp các ứng dụng AI trở nên:

Kết nối với các nguồn dữ liệu bên ngoài: Giúp LLM có thể truy cập và làm việc với các bộ dữ liệu riêng tư hoặc thông tin mới mà nó chưa từng được huấn luyện, đây chính là nền tảng cho các ứng dụng ChatBot trên bộ dữ liệu cá nhân.

Tương tác với môi trường: Cho phép LLM không chỉ trả lời một cách thụ động mà còn có thể sử dụng các công cụ để thực hiện hành động, từ đó hoàn thành các nhiệm vụ phức tạp hơn.



Hình 18: Cách thức hoạt động của LangChain

Chương 3. XÂY DỰNG HỆ THỐNG TÁC TỬ AI TRONG NÔNG NGHIỆP THÔNG MINH ỨNG DỤNG RAG

3.1. Giới thiệu hệ thống

Đề án “**Ứng dụng RAG xây dựng hệ thống tác tử AI trong nông nghiệp thông minh**” tập trung vào việc nghiên cứu, xây dựng và triển khai một hệ thống **tác tử AI (AI Agent)** hoàn chỉnh. Hệ thống ứng dụng **Mô hình Ngôn ngữ lớn (LLM – Large Language Model)** kết hợp với kiến trúc **RAG (Retrieval-Augmented Generation)** nhằm cung cấp kiến thức chuyên sâu và hỗ trợ ra quyết định trong bối cảnh nông nghiệp thông minh.

Mục tiêu chính của đề án là phát triển một tác tử AI với giao diện người dùng thân thiện. Tác tử này sở hữu năng lực cung cấp câu trả lời chuyên môn với độ chính xác cao dựa trên một cơ sở tri thức đã được kiểm duyệt, đồng thời có khả năng sử dụng các công cụ chuyên dụng như truy vấn dữ liệu thời tiết theo thời gian thực và hỗ trợ đưa ra các khuyến nghị phù hợp.

Đề án này tập trung vào việc xây dựng tác tử AI mạnh mẽ, giải quyết vấn đề khó khăn trong việc tiếp cận thông tin kỹ thuật của người nông dân. Bằng cách cung cấp một kênh truy vấn tập trung, đáng tin cậy và có khả năng suy luận, hệ thống góp phần nâng cao năng suất và giảm thiểu rủi ro do thiếu thông tin.

3.2. Các chức năng chính của hệ thống

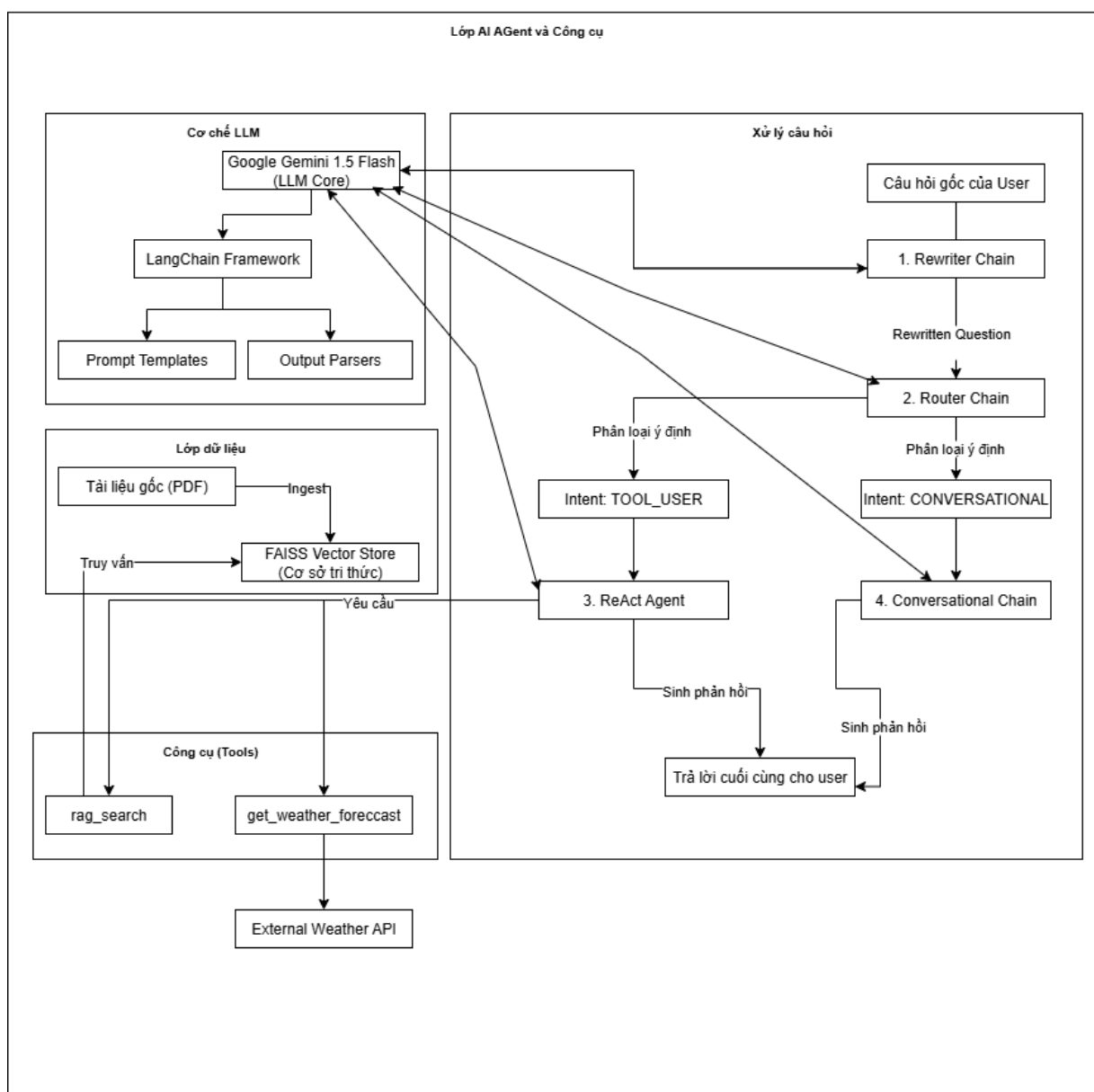
Hệ thống tác tử được thiết kế với các chức năng cốt lõi sau:

- **Hệ thống phân tích và định tuyến câu hỏi:** Hệ thống có khả năng phân biệt giữa các tương tác giao tiếp thông thường và các yêu cầu thông tin chuyên  môn, thời tiết từ đó lựa chọn công cụ và phương thức xử lý phù hợp.

- **Tra cứu và trả lời dựa trên cơ sở tri thức (RAG):** Khi nhận câu hỏi chuyên môn, hệ thống sẽ truy xuất các thông tin liên quan từ một **cơ sở tri thức (knowledge base)** nông nghiệp được xây dựng bằng cách xử lý và chuẩn hóa các tài liệu nông nghiệp uy tín (*như giáo trình, tài liệu kỹ thuật, quy trình chuẩn,...*). Sau đó, những thông tin này sẽ được đưa vào một **lời nhắc (prompt)** mới cùng với câu hỏi gốc để **Mô hình Gemini** tổng hợp, diễn giải và tạo ra một câu trả lời cuối cùng mạch lạc, dễ hiểu cho người dùng, đồng thời vẫn đảm bảo tính chính xác và khả năng truy xuất nguồn.

- **Tích hợp công cụ thời tiết thời gian thực:** Cung cấp dự báo thời tiết tại một địa điểm cụ thể và nếu người dùng hỏi thì có thể đưa ra các khuyến nghị liên quan, ví dụ như có nên tưới tiêu hay không dựa trên dự báo mưa.
- **Quản lý ngữ cảnh và lịch sử hội thoại:** Hệ thống có khả năng ghi nhớ các trao đổi trước đó trong cùng một phiên làm việc, giúp người dùng có thể đặt các câu hỏi nối tiếp một cách tự nhiên.
- **Quản lý người dùng và lịch sử trò chuyện:** Cho phép người dùng đăng ký, đăng nhập để lưu trữ và xem lại các cuộc hội thoại trước đây của mình.

3.3. Kiến trúc hệ thống



Hình 19: Tổng quan kiến trúc hệ thống

3.4. Luồng hoạt động của hệ thống

Hệ thống tác tử có hai luồng hoạt động chính, hoạt động độc lập với nhau: **Luồng xử lý dữ liệu** và **luồng trả lời câu hỏi** (thực hiện theo thời gian thực).

3.4.1. Luồng xử lý dữ liệu (Xây dựng Cơ sở tri thức)

Đây là luồng công việc được thực hiện trước, nhằm chuẩn bị cho “bộ não” kiến thức cho tác tử AI. Luồng này biến các tài liệu nông nghiệp từ dạng tệp PDF thành một cơ sở dữ liệu vector có cấu trúc.

Thu thập dữ liệu: Các tệp tài liệu nông nghiệp ở định dạng .pdf được thu thập và lưu vào thư mục data/raw.

Trích xuất văn bản: Hệ thống tự động quét qua từng tệp PDF. Nó đầu tiên cố gắng trích xuất văn bản trực tiếp. Nếu không thành công (thường là với file PDF dạng ảnh scan), hệ thống sẽ chuyển sang sử dụng công nghệ nhận dạng ký tự quang học (OCR) với Google Vision hoặc Tesseract để chuyển đổi hình ảnh thành văn bản.

Làm sạch và phân đoạn: Văn bản thô sau khi được trích xuất sẽ được làm sạch (sửa lỗi OCR, chuẩn hóa khoảng trắng). Sau đó, thư viện *RecursiveCharacterTextSplitter* của LangChain sẽ chia toàn bộ văn bản thành các đoạn nhỏ hơn (chunks) có độ dài khoảng 1000 ký tự và gộp lên nhau 200 ký tự để đảm bảo không mất ngữ cảnh.

Vector hóa (Embedding): Mỗi đoạn văn bản (chunk) sẽ được đưa qua mô hình embedding *sentences-transformers*. Mô hình này sẽ chuyển mỗi chunk thành một vector số học đại diện cho ý nghĩa ngữ nghĩa của nó.

Lập chỉ mục (Indexing): Cuối cùng, tất cả các cặp (đoạn văn bản, vector tương ứng) được lưu trữ và lập chỉ mục trong một cơ sở dữ liệu vector sử dụng **FAISS**. File chỉ mục này được lưu tại indexes/faiss_index và sẵn sàng cho việc truy vấn ở tốc độ cao.

3.4.2. Luồng trả lời câu hỏi (Tương tác thời gian thực)

Đây là luồng hoạt động chính diễn ra khi người dùng tương tác với tác tử AI trên giao diện web.

Nhận yêu cầu: Người dùng nhập câu hỏi vào giao diện chat và nhấn gửi. Yêu cầu này được gửi đến một endpoint API trên Backend Flask.

Xác thực và lấy ngữ cảnh: Hệ thống xác thực người dùng đã đăng nhập và tải lịch sử của phiên trò chuyện hiện tại từ cơ sở dữ liệu MySQL để làm ngữ cảnh.

Phân tích và định tuyến:

Rewrite Chain: Câu hỏi của người dùng được đưa qua một chuỗi xử lý để biến nó thành một câu hỏi độc lập, đầy đủ ngữ cảnh hơn.

Router Chain: Câu hỏi đã được viết lại sau đó được phân loại thành **TOOL_USER** (cần dùng công cụ) và **CONVERSATIONAL** (trò chuyện thông thường).

Thực thi Tác tử:

Nếu là **CONVERSATIONAL**, một chuỗi xử lý đơn giản sẽ gọi **LLM Gemini** để tạo ra câu trả lời xã giao.

Nếu là **TOOL_USER**, tác tử ReAct sẽ được kích hoạt. Tác tử sẽ suy luận và quyết định sử dụng dụng cụ phù hợp (*rag_search* hoặc *get_weather_forecast*). Nếu dùng *rag_search*, toàn bộ quy trình RAG (truy xuất, tái xếp hạng) sẽ được thực thi để lấy ngữ cảnh.

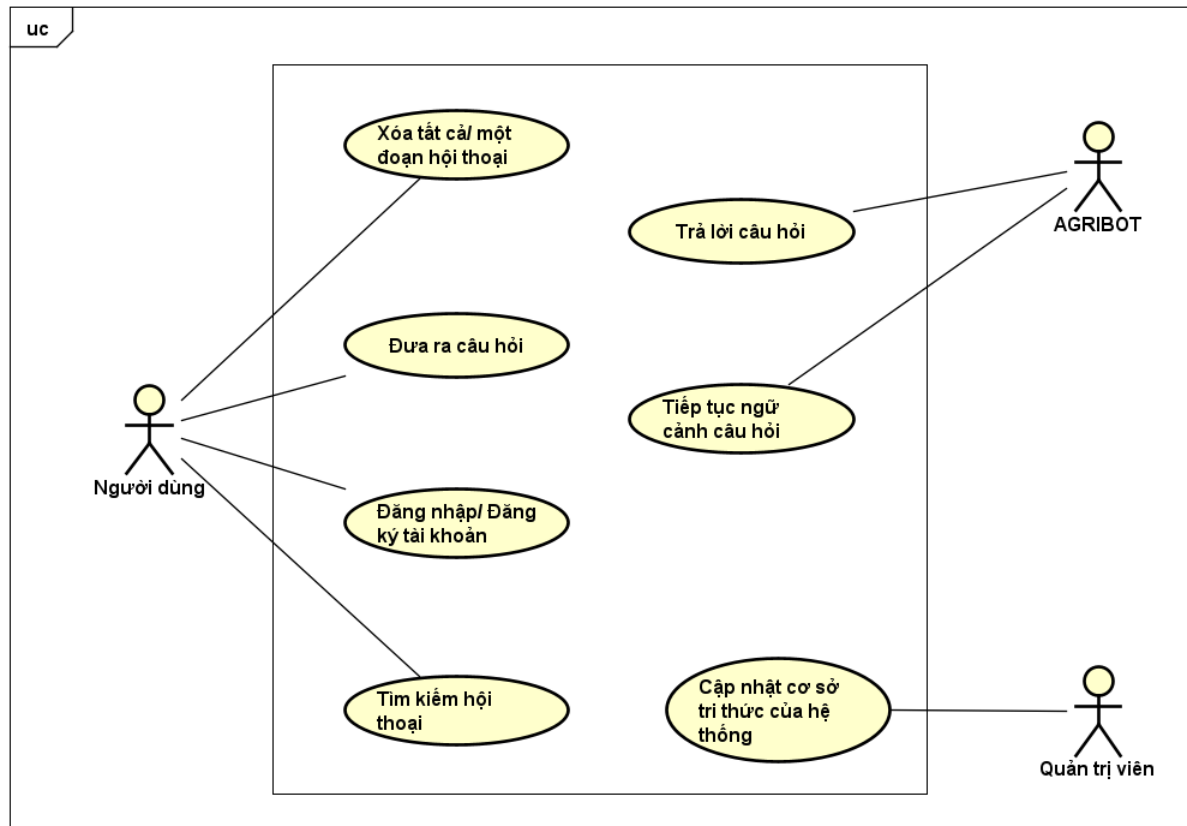
Tạo sinh và trả lời: Tác tử tổng hợp tất cả các thông tin (kết quả từ công cụ, lịch sử trò chuyện) và gọi **LLM Gemini** một lần cuối để tạo ra câu trả lời hoàn chỉnh.

Lưu trữ: Câu hỏi của người dùng và câu trả lời cuối cùng của AgriBot được lưu vào cơ sở dữ liệu MySQL.

Phản hồi: Câu trả lời và các nguồn tham khảo (nếu có) được gửi đến Frontend để hiển thị cho người dùng.

3.5. Phân tích hệ thống

3.5.1. Lược đồ use case hệ thống AgriBot



Hình 20: Lược đồ use case của hệ thống

3.5.2. Đặc tả chức năng use case đăng ký người dùng

Mã Use Case	UC01
Tên Use Case	Đăng ký tài khoản
Mô tả Use case	Cho phép người dùng mới tạo một tài khoản để truy cập vào hệ thống
Actor chính	Người dùng
Actor phụ	
Tiền điều kiện	Người dùng chưa có tài khoản và chưa đăng nhập.
Hậu điều kiện	1. Một tài khoản mới được tạo trong cơ sở dữ liệu. 2. Người dùng được tự động đăng nhập và chuyển tới trang chat chính.

Luồng hoạt động chính	1. Người dùng truy cập trang web và chọn đăng ký 2. Hệ thống hiển thị giao diện đăng ký. 3. Người dùng nhập tên đăng nhập và mật khẩu (và cả gmail nếu muốn). 4. Người dùng nhấn nút tạo tài khoản. 5. Hệ thống kiểm tra thông tin hợp lệ (độ dài, tên đăng nhập chưa tồn tại). 6. Hệ thống tạo tài khoản mới và lưu vào cơ sở dữ liệu. 7. Hệ thống tự động đăng nhập và chuyển hướng người dùng đến đoạn chat.
Luồng thay thế	5a. Tên đăng nhập đã tồn tại: Hệ thống báo lỗi và yêu cầu người dùng chọn tên khác.
Luồng xử lý ngoại lệ	

Bảng 1: Bảng đặc tả chức năng use case đăng ký người dùng

3.5.3. Đặc tả chức năng use case đăng nhập người dùng

Mã Use Case	UC02
Tên Use Case	Đăng nhập
Mô tả Use case	Cho phép người dùng đã có tài khoản truy cập vào hệ thống.
Actor chính	Người dùng
Actor phụ	
Tiền điều kiện	Người dùng đã có tài khoản và chưa thực hiện đăng nhập
Hậu điều kiện	Người dùng được xác thực thành công và chuyển đến trang chat chính.
Luồng hoạt động chính	1. Người dùng truy cập trang web 2. Hệ thống hiển thị giao diện đăng nhập 3. Người dùng nhập tên đăng nhập và mật khẩu

	4. Người dùng nhấn nút “Đăng nhập” 5. Hệ thống xác thực thông tin với cơ sở dữ liệu. 6. Hệ thống tạo phiên làm việc và chuyển hướng người dùng đến trang chat.
Luồng thay thế	5a. Sai tên đăng nhập hoặc mật khẩu: Hệ thống báo lỗi
Luồng xử lý ngoại lệ	

Bảng 2: Bảng đặc tả chức năng use case đăng nhập

3.5.4. Đặc tả chức năng use case đưa ra câu hỏi

Mã Use Case	UC03
Tên Use Case	Đưa ra câu hỏi
Mô tả Use case	Cho phép Người dùng nhập và gửi một câu hỏi hoặc một yêu cầu cho hệ thống.
Actor chính	Người dùng
Actor phụ	
Tiền điều kiện	Người dùng đã đăng nhập thành công
Hậu điều kiện	Câu hỏi của Người dùng được ghi nhận và gửi đến hệ thống xử lý.
Luồng hoạt động chính	1. Người dùng nhập nội dung câu hỏi vào ô chat trên giao diện. 2. Người dùng thực hiện hành động gửi 3. Hệ thống tiếp nhận câu hỏi
Luồng thay thế	2a. Gửi tin nhắn trống: Nếu người dùng nhấn gửi khi chưa nhập nội dung, hệ thống sẽ không thực hiện hành động nào.
Luồng xử lý ngoại lệ	3a. Lỗi kết nối mạng: Nếu không có kết nối mạng, yêu cầu của Người dùng sẽ thất bại và giao diện người dùng hiển thị lỗi tương ứng

Bảng 3: Bảng đặc tả chức năng use case đưa ra câu hỏi

3.5.5. Đặc tả chức năng use case trả lời câu hỏi

Mã Use Case	UC04
Tên Use Case	Trả lời câu hỏi
Mô tả Use case	Hệ thống xử lý câu hỏi của người dùng và tạo ra một câu trả lời phù hợp
Actor chính	Hệ thống
Actor phụ	Người dùng
Tiền điều kiện	Hệ thống đã nhận được một câu hỏi từ Người dùng (kết quả từ UC03)
Hậu điều kiện	Một câu trả lời được tạo ra và hiển thị cho người dùng
Luồng hoạt động chính	1. Hệ thống phân tích câu hỏi đã nhận 2. Hệ thống truy xuất thông tin liên quan từ cơ sở tri thức (RAG) 3. Hệ thống sử dụng LLM để tổng hợp thông tin và tạo ra câu trả lời. 4. Hệ thống hiển thị câu trả lời lên giao diện của Người dùng.
Luồng thay thế	2a. Câu hỏi trò chuyện thông thường: Hệ thống xác định câu hỏi không cần kiến thức chuyên môn, bỏ qua bước 2(truy xuất RAG) và đi thẳng đến bước 3 để tạo câu trả lời xã giao.
Luồng xử lý ngoại lệ	2b. Không tìm thấy thông tin: Hệ thống không tìm thấy tài liệu liên quan trong cơ sở tri thức. Hệ thống sẽ tạo ra một câu trả lời rằng không có thông tin và hiển thị cho Người dùng. Usecase kết thúc. E1. Lỗi API LLM: Nếu có lỗi kết nối đến LLM, hệ thống sẽ hiển thị thông báo lỗi chung.

Bảng 4: Bảng đặc tả chức năng use case trả lời câu hỏi

3.5.6. Đặc tả chức năng use case tiếp tục ngữ cảnh câu hỏi

Mã Use Case	UC05
Tên Use Case	Tiếp tục ngữ cảnh câu hỏi
Mô tả Use case	Hệ thống duy trì và sử dụng lịch sử của cuộc hội thoại để hiểu và trả lời các câu hỏi nối tiếp.
Actor chính	Hệ thống
Actor phụ	Người dùng
Tiền điều kiện	Cuộc hội thoại đã có ít nhất một cặp hỏi – đáp trước đó.
Hậu điều kiện	Câu trả lời cho câu hỏi nối tiếp có tính đến các thông tin đã trao đổi trước đó.
Luồng hoạt động chính	1. Hệ thống nhận câu hỏi nối tiếp từ Người dùng 2. Hệ thống tải lịch sử của cuộc hội thoại hiện tại 3. Hệ thống sử dụng lịch sử để diễn giải câu hỏi nối tiếp thành một câu hỏi đầy đủ ngữ cảnh 4. Hệ thống tiếp tục thực hiện UC04 với câu hỏi đã được làm rõ.
Luồng thay thế	1a. Câu hỏi không liên quan đến ngữ cảnh: Nếu câu hỏi nối tiếp của Người dùng là một chủ đề hoàn toàn mới, hệ thống vẫn sẽ xử lý như một câu hỏi độc lập bình thường.
Luồng xử lý ngoại lệ	2a. Lỗi tải lịch sử: Nếu không thể tải được lịch sử trò chuyện, hệ thống sẽ xử lý câu hỏi như một câu hỏi mới, độc lập.

Bảng 5: Bảng đặc tả chức năng use case tiếp tục ngữ cảnh câu hỏi

3.5.7. Đặc tả chức năng use case tìm kiếm hội thoại

Mã Use Case	UC06
Tên Use Case	Tìm kiếm hội thoại
Mô tả Use case	Cho phép Người dùng nhập tên hội thoại mà mình muốn tìm

Actor chính	Người dùng
Actor phụ	
Tiền điều kiện	Người dùng đã đăng nhập và có lịch sử hội thoại.
Hậu điều kiện	Danh sách hội thoại trên giao diện được thu hẹp theo từ khóa tìm kiếm.
Luồng hoạt động chính	1. Người dùng nhập từ khóa vào ô tìm kiếm hội thoại 2. Hệ thống lọc danh sách hội thoại trên giao diện chỉ hiển thị những hội thoại có tiêu đề khớp với từ khóa.
Luồng thay thế	2a. Không có kết quả: Nếu không có hội thoại nào khớp, giao diện sẽ hiển thị thông báo “Không tìm thấy kết quả”.
Luồng xử lý ngoại lệ	

Bảng 6: Bảng đặc tả chức năng use case tìm kiếm đoạn hội thoại

3.5.8. Đặc tả chức năng use case xóa hội thoại

Mã Use Case	UC07
Tên Use Case	Xóa hội thoại
Mô tả Use case	Cho phép Người dùng xóa vĩnh viễn một hoặc toàn bộ các đoạn hội thoại.
Actor chính	Người dùng
Actor phụ	
Tiền điều kiện	Người dùng đã đăng nhập và có ít nhất một cuộc hội thoại.
Hậu điều kiện	Cuộc hội thoại được chọn sẽ bị xóa khỏi hệ thống.
Luồng hoạt động chính	1. Người dùng chọn hành động xóa trên một đoạn hội thoại cụ thể hoặc nút “<i>Xóa tất cả hội thoại</i>” 2. Hệ thống yêu cầu xác nhận 3. Người dùng xác nhận

	4. Hệ thống xóa vĩnh viễn cuộc hội thoại và các tin nhắn liên quan khỏi cơ sở dữ liệu. 5. Hệ thống cập nhật lại danh sách hội thoại trên giao diện.
Luồng thay thế	3a. Người dùng hủy bỏ: Tại bước 3, nếu Người dùng chọn “Hủy”, usecase sẽ kết thúc mà không có hành động nào được thực hiện.
Luồng xử lý ngoại lệ	4a. Lỗi server/ kết nối: Nếu hệ thống không thể xóa hội thoại do lỗi kết nối, một thông báo lỗi sẽ được hiển thị.

Bảng 7: Bảng đặc tả chức năng use case xóa hội thoại

3.5.9. Đặc tả chức năng use case cập nhật cơ sở tri thức hệ thống

Mã Use Case	UC08
Tên Use Case	Cập nhật cơ sở tri thức của hệ thống
Mô tả Use case	Cho phép Quản trị viên thêm/bớt/sửa các tài liệu nguồn để cập nhật kiến thức cho AgriBot.
Actor chính	Quản trị viên
Actor phụ	
Tiền điều kiện	Quản trị viên đã đăng nhập với quyền hạn tương ứng.
Hậu điều kiện	Cơ sở tri thức của hệ thống được cập nhật.
Luồng hoạt động chính	1. Quản trị viên thay đổi các tệp tài liệu trong thư mục dữ liệu nguồn. 2. Quản trị viên chạy kịch bản xử lý dữ liệu (ingest.py) 3. Hệ thống xử lý các tài liệu thành định dạng chuẩn 4. Quản trị viên chạy kịch bản lập chỉ mục (indexing.py) 5. Hệ thống tạo lại chỉ mục vector từ dữ liệu đã được xử lý.
Luồng thay thế	

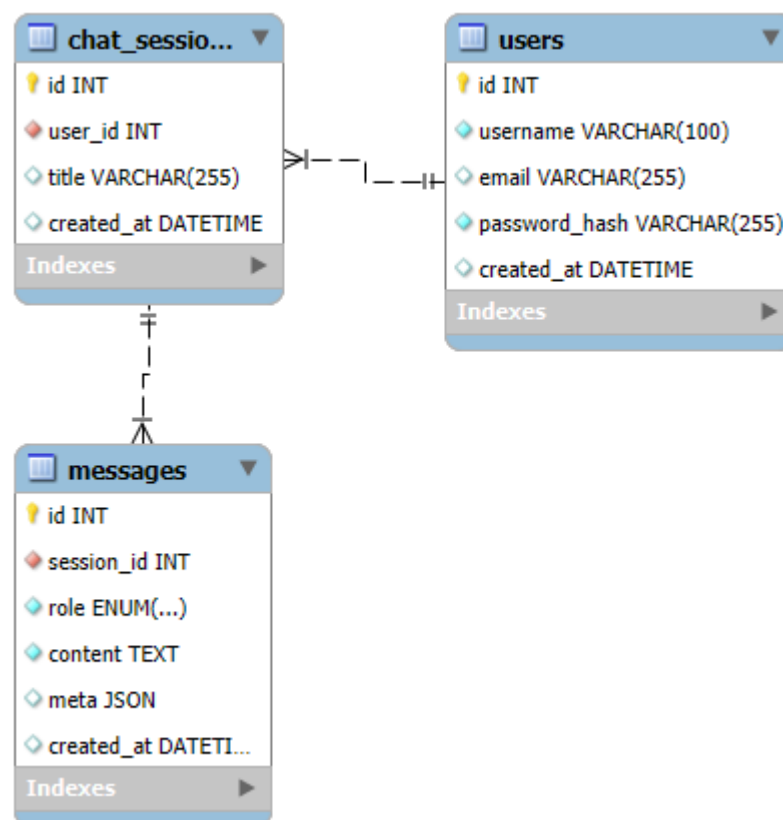
Luồng xử lý ngoại lệ	3a. Lỗi xử lý tệp PDF: Nếu một tệp tài liệu bị lỗi, ghi nhận lỗi và tiếp tục xử lý các tệp còn lại. 5ba. Lỗi lập chỉ mục: Nếu có lỗi trong quá trình lập chỉ mục, hệ thống sẽ dừng lại và báo lỗi cho Quản trị viên
----------------------	---

Bảng 8: Bảng đặc tả chức năng use cập nhật cơ sở tri thức hệ thống

3.6. Thiết kế cơ sở dữ liệu

Hệ thống sử dụng mô hình cơ sở dữ liệu quan hệ để lưu trữ dữ liệu một cách bền vững. Lựa chọn này đảm bảo tính toàn vẹn, nhất quán và khả năng truy vấn dữ liệu hiệu quả. Cơ sở dữ liệu được triển khai bằng **MySQL** và được tương tác thông qua **SQLAlchemy ORM (Object-Relational Mapping)**, giúp ánh xạ các đối tượng Python vào các bảng trong cơ sở dữ liệu.

3.6.1. Lược đồ ảo trực quan hóa cơ sở dữ liệu



Hình 21: Lược đồ ảo trực quan hóa CSDL

3.6.2. Thiết kế chi tiết các bảng

3.6.2.1. Bảng users

Đây là nơi được lưu trữ những thông tin tài khoản của người dùng

Tên cột	Kiểu dữ liệu	Mô tả
id	INTERGER	Khóa chính, tự động tăng
username	VARCHAR(100)	Tên đăng nhập của người dùng, duy nhất, không được rỗng
email	VARCHAR(255)	Email của người dùng, duy nhất, có thể rỗng
password_hash	VARCHAR(255)	Chuỗi băm mật khẩu, không được rỗng
created_at	DATETIME	Mốc thời gian tài khoản được tạo

Bảng 9: Chi tiết bảng users

3.6.2.2. Bảng chat_sessions

Bảng này lưu trữ thông tin về mỗi cuộc hội thoại

Tên cột	Kiểu dữ liệu	Mô tả
id	INTERGER	Khóa chính, tự động tăng.
user_id	INTERGER	Khóa ngoại, liên kết đến users.id. Cho biết phiên chat này thuộc về người dùng nào.
title	VARCHAR(255)	Tiêu đề của cuộc hội thoại
created-at	DATETIME	Mốc thời gian phiên chat được tạo

Bảng 10: Chi tiết bảng chat_sessions

3.6.2.3. Bảng messages

Bảng này lưu trữ nội dung của từng tin nhắn trong các cuộc hội thoại

Tên cột	Kiểu dữ liệu	Mô tả
id	INTERGER	Khóa chính, tự động tăng.
session_id	INTERGER	Khóa ngoại, liên kết đến chat_sessions.id. Cho biết tin nhắn này thuộc phiên chat nào.
role	ENUM(“user”, “assistant”)	Vai trò của người gửi tin nhắn
content	TEXT	Nội dung chính của tin nhắn
meta	JSON	Một trường linh hoạt để lưu các thông tin phụ, ví dụ như danh sách các nguồn tham khảo cho câu trả lời của AI. Có thể rỗng.
created-at	DATETIME	Mốc thời gian tin nhắn được tạo

Bảng 11: Chi tiết bảng messages

3.7. Thiết kế hệ thống

3.7.1. Tổng quan kiến trúc AI Agents

Hệ thống được xây dựng dựa trên nguyên lý hoạt động của **tác tử AI (AI Agent)**, một cách tiếp cận tiên tiến cho phép hệ thống không chỉ trả lời các câu hỏi mà còn có khả năng suy luận, lập kế hoạch và thực hiện các hành động phức tạp. Kiến trúc cốt lõi được áp dụng cho AgriBot là mô hình **ReAct (Reason + Act)**, kết hợp khả năng suy luận của mô hình LLM với khả năng thực hiện hành động thông qua các công cụ.

Vòng lặp hoạt động của một tác tử ReAct trong hệ thống được diễn ra như sau:

Thought (Suy nghĩ): LLM phân tích yêu cầu của người dùng, xem xét lịch sử trò chuyện và danh sách các công cụ có sẵn để quyết định bước tiếp theo làm gì.

Action (Hành động): Tác tử chọn một công cụ phù hợp từ bộ công cụ (AGRIBOT_TOOLBOX) và xác định đầu vào cho nó.

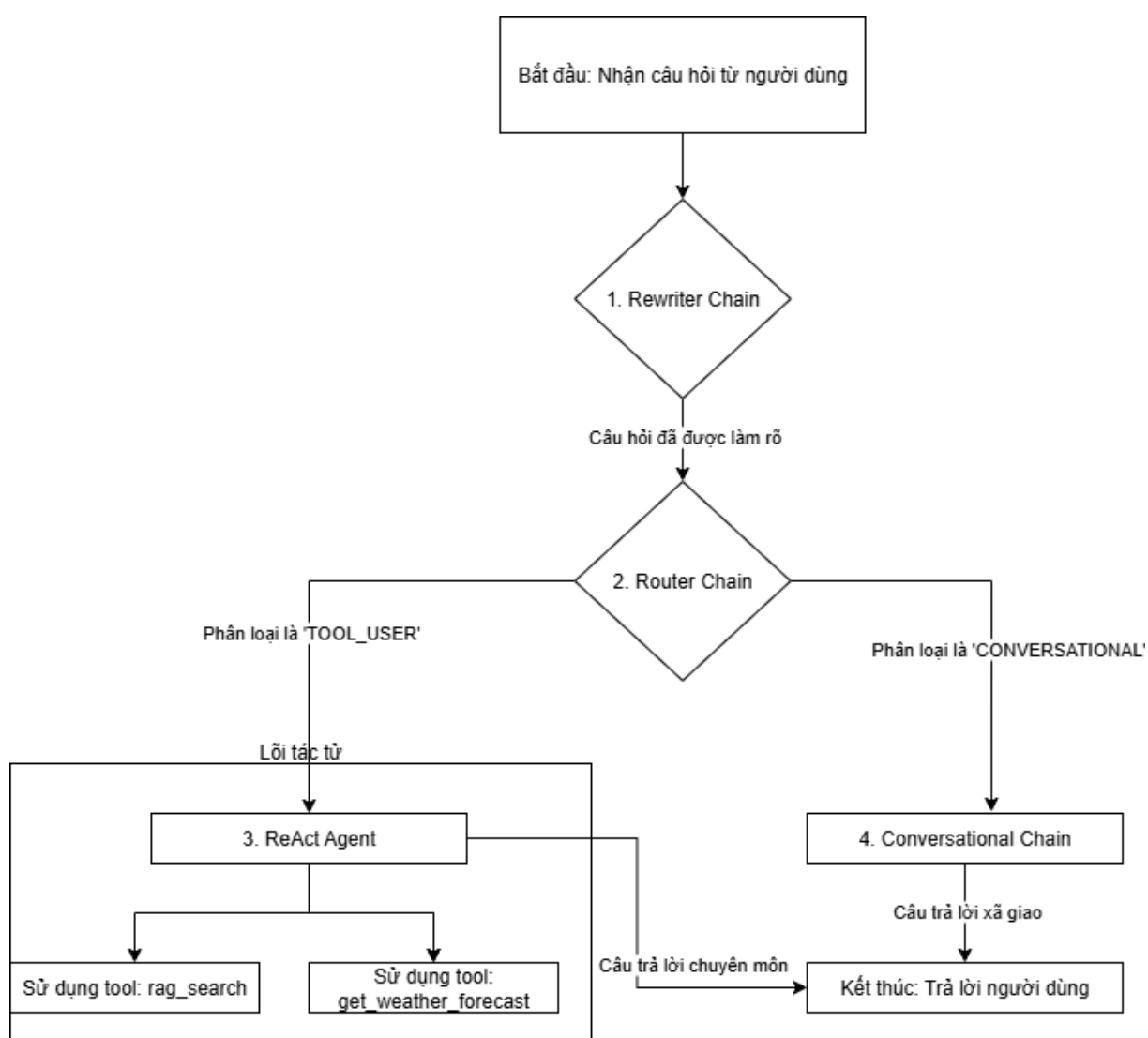
Observation (Quan sát): Hệ thống thực thi công cụ và trả về kết quả cho tác tử.

Lặp lại: Tác tử tích hợp quan sát mới và quay lại bước “*Thought*” để tiếp tục suy luận cho đến khi có đủ thông tin để đưa ra câu trả lời cuối cùng (Final Answer).

Kiến trúc này giúp hệ thống giải quyết các yêu cầu đa dạng, tăng tính minh bạch và linh hoạt cho việc mở rộng.

3.7.2. Phân tích chi tiết các thành phần trong hệ thống tác tử

Kiến trúc tác tử của AgriBot được triển khai như một pipeline xử lý gồm nhiều giai đoạn, cho phép hệ thống phân tích và định tuyến yêu cầu của người dùng một cách hiệu quả.



Hình 22: Luồng xử lý từ khi nhận câu hỏi đến khi trả lời

3.7.2.1. Chuỗi viết lại câu hỏi (Rewriter Chain)

Mục đích: Xử lý ngữ cảnh của cuộc trò chuyện, biến các câu hỏi nối tiếp (ví dụ: “còn cây lúa thì sao?”) thành một câu hỏi độc lập, đầy đủ ngữ nghĩa (“cách bón phân cho cây lúa thì sao?”) dựa trên lịch sử hội thoại trước đó.

Chi tiết kỹ thuật: Một *ChatPromptTemplate* được tạo ra, chứa một chỉ thị hệ thống yêu cầu LLM biến câu hỏi nối tiếp thành câu hỏi độc lập. Chuỗi này được hoàn thiện bằng cách kết nối prompt với mô hình LLM và một *StrOutputParser*.

```
rewriter_system_message = """Dựa vào lịch sử trò chuyện, hãy biến câu hỏi nối tiếp của người dùng thành một câu hỏi đầy đủ, độc lập.
QUY TẮC QUAN TRỌNG:
- Nếu câu hỏi của người dùng đã đầy đủ, hãy giữ nguyên.
- Nếu đầu vào của người dùng không phải là một câu hỏi (ví dụ: là lời chào, lời cảm ơn, một câu khẳng định), HÃY TRẢ VỀ Y NGUYÊN ĐẦU VÀO ĐÓ MÀ KHÔNG THAY ĐỔI GÌ."""

rewriter_prompt = ChatPromptTemplate.from_messages([
    [
        ("system", rewriter_system_message),
        ("user", "Lịch sử trò chuyện:\n{chat_history}\n\nCâu hỏi nối tiếp: {input}\n\nCâu hỏi đầy đủ:")
    ]
])

REWRITER_CHAIN = rewriter_prompt | LLM_SYSTEM | StrOutputParser()
```

Hình 23: Mã nguồn cho Rewriter Chain

3.7.2.2. Chuỗi định tuyến (Router Chain)

Mục đích: Hoạt động như một “bộ điều phối”, phân loại ý định của người dùng để lựa chọn luồng xử lý phù hợp, tránh lãng phí tài nguyên. Việc kích hoạt tác tử ReAct đầy đủ cho các câu hỏi đơn giản như “Cảm ơn” sẽ rất tốn kém và chậm chạp. Router Chain giúp hệ thống phản hồi nhanh chóng cho các tương tác xã giao và chỉ huy động tác tử mạnh mẽ khi thực sự cần thiết.

Chi tiết kỹ thuật: Một *ChatPromptTemplate* được xây dựng từ một chuỗi văn bản chi tiết, giải thích rõ hai loại ý định (‘TOOL_USER’ và ‘CONVERSATIONAL’) kèm theo ví dụ cụ thể. Chuỗi này cũng được kết nối và parser để trả về một trong hai nhãn phân loại.

```

router_prompt_text = ""Bạn là một bộ định tuyến (router) thông minh, có nhiệm vụ phân loại câu hỏi của người dùng thành một trong hai loại sau: 'TOOL_USER' hoặc 'CONVERSATIONAL'.

Các quy tắc phân loại như sau:

1. **TOOL_USER**: Phân loại là 'TOOL_USER' nếu câu hỏi yêu cầu thông tin thực tế, kiến thức chuyên môn về nông nghiệp hoặc dự báo thời tiết. Đây là những câu hỏi cần sử dụng công cụ để tra cứu.
- Ví dụ: "Sầu đay thì là gì?", "Cách bón phân cho cây lúa", "Tiêu chuẩn VietGAP yêu cầu gì về bảo quản?", "Thời tiết ngày mai ở Cần Thơ".

2. **CONVERSATIONAL**: Phân loại là 'CONVERSATIONAL' nếu câu hỏi là lời chào hỏi, cảm ơn, câu hỏi về bản thân con bot.
- Nếu người dùng hỏi câu hỏi không liên quan đến phạm trù NÔNG NGHIỆP thì trả lời một cách lịch sự bạn là một AI hỗ trợ nông nghiệp, không trả lời những câu hỏi ngoài nông nghiệp
- Ví dụ: "Chào bạn", "Bạn là ai?", "Cảm ơn nhé", "Bạn có khỏe không?".

Dựa vào lịch sử trò chuyện và câu hỏi mới nhất, hãy thực hiện phân loại. Chỉ trả lời bằng 'TOOL_USER' hoặc 'CONVERSATIONAL'.

Lịch sử trò chuyện:
{chat_history}

Câu hỏi mới nhất: {input}
Phân loại:"""

router_prompt = ChatPromptTemplate.from_template(router_prompt_text)
ROUTER_CHAIN = router_prompt | LLM_SYSTEM | StrOutputParser()

```

Hình 24: Mã nguồn cho Router Chain

3.7.2.3. Tác tử chính (ReAct Agent – Xử lý chuyên môn)

Mục đích: Là thành phần mạnh mẽ nhất, xử lý tất cả các yêu cầu phức tạp đòi hỏi kiến thức chuyên môn và sử dụng công cụ. Được thiết kế để xử lý các câu hỏi “đuôi dài” (long-tail) mà một hệ thống RAG đơn giản không thể trả lời, ví dụ các câu hỏi đòi hỏi thông tin kết hợp từ nhiều nguồn (kiến thức nội bộ và API thời tiết).

Chi tiết kỹ thuật: Các hàm Python được gói lại thành các đối tượng Tool của LangChain. ‘*REACT_PROMPT_TEMPLATE*’ được tạo ra để chỉ dẫn LLM về vai trò và định dạng vòng lặp ReAct. Hàm ‘*create_react_agent*’ của LangChain được sử dụng để kết hợp LLM, công cụ và prompt. Tác tử này sau đó được đưa vào một ‘*AgentExecutor*’ để thực thi.

```

# 3. Khởi tạo Agent Executor (dùng cho các tác vụ phức tạp cần tool)
rag_tool = get_rag_tool()
all_tools = AGRIBOT_TOOLBOX + [rag_tool]
agent = create_react_agent(LLM_SYSTEM, all_tools, PROMPT)
AGENT_EXECUTOR = AgentExecutor(
    agent=agent,
    tools=all_tools,
    verbose=True,
    handle_parsing_errors=True,
    return_intermediate_steps=True,
)

```

Hình 25: Mã nguồn cho ReAct Agent

3.7.2.4. Chuỗi hội thoại (Conversational Chain – Xử lý xã giao)

Mục đích: Xử lý các tương tác thân thiện, lịch sự và đúng vai trò cho các yêu cầu không mang tính chuyên môn. Nó cũng có vai trò quan trọng trong việc lịch sự từ chối các câu

hỏi nằm ngoài phạm vi nông nghiệp, giữ cho hệ thống tập trung vào đúng chuyên môn của mình.

Chi tiết kỹ thuật: Một ‘*conversational_system_message*’ riêng được định nghĩa để hướng dẫn LLM cách trả lời thân thiện. Chuỗi này được tạo và gọi linh hoạt bên trong hàm ‘*api_chat*’ khi ‘ROUTER_CHAIN’ xác định là ‘CONVERSATIONAL’.

```
if "CONVERSATIONAL" in intent:
    conversational_system_message = """Bạn là AgriBot, một trợ lý AI chuyên về nông nghiệp.
    QUY TẮC TRẢ LỜI:
    1. Nếu người dùng chào, hỏi thăm hoặc cảm ơn (ví dụ: "chào bạn", "cảm ơn", "bạn khỏe không"), hãy đáp lại một cách lịch sự, thân thiện.
    2. Nếu người dùng hỏi một câu hỏi không liên quan đến nông nghiệp (ví dụ: "Kể chuyện cười đi", "Thủ đô của Mỹ là gì?", "Viết một bài thơ"),
    hãy trả lời chính xác như sau: "Tôi là AgriBot, một trợ lý AI chuyên về nông nghiệp. Tôi chỉ có thể hỗ trợ các câu hỏi liên quan đến lĩnh vực này."
    3. Tuyệt đối không trả lời các câu hỏi nằm ngoài chuyên môn nông nghiệp.
    """
    conv_prompt = ChatPromptTemplate.from_messages(
        [
            ("system", conversational_system_message),
            ("user", "{input}")
        ]
    )
    conv_chain = conv_prompt | llm | StrOutputParser()
    final_answer = conv_chain.invoke({"input": rewritten_question})
else: # "TOOL_USER"
    result = agent_executor.invoke({
        "input": rewritten_question,
        "chat_history": memory.chat_memory.messages
    })
    final_answer = result.get("output", "Rất tiếc, tôi chưa tìm ra câu trả lời.")
```

Hình 26: Mã nguồn cho Conversational Chain

3.7.3. Thiết kế Prompt (Prompt Engineering)

Trong các hệ thống dựa trên LLM, prompt đóng vai trò như mã nguồn để điều khiển hành vi của AI. Việc thiết kế prompt một cách cẩn thận là yếu tố quyết định đến hiệu suất và độ tin cậy của hệ thống.

3.7.3.1. Nguyên tắc thiết kế Prompt chung

Xác định vai trò: Gán cho AI một vai trò cụ thể (“Bạn là AgriBot...”).

Cung cấp chỉ thị rõ ràng: Đưa ra các mệnh lệnh trực tiếp, cụ thể.

Cung cấp ví dụ: Đưa ra các ví dụ cụ thể để LLM học theo.

Xác định định dạng đầu ra: Yêu cầu LLM trả lời theo một cấu trúc nhất định.

3.7.3.2. Phân tích các Prompt chính trong hệ thống

Đây là prompt phức tạp và quan trọng nhất, đóng vai trò như “bộ luật” và “sách hướng dẫn” cho toàn bộ quá trình suy luận của AgriBot. Mỗi thành phần trong prompt này đều được thiết kế cẩn thận để kiểm soát và định hướng hành vi của LLM.

```

Bạn là AgriBot, một trợ lý nông nghiệp thân thiện và giàu kinh nghiệm.
Mục tiêu của bạn là cung cấp thông tin chính xác, đáng tin cậy và dễ hiểu. Bạn hiện đang ở chế độ sử dụng công cụ.

TOOLS YOU CAN USE:
{tools}

POLICY (read carefully):
- Review the conversation history để hiểu rõ bối cảnh câu hỏi của người dùng.
- Nếu câu hỏi về nông nghiệp (cây trồng, bệnh hại, kỹ thuật), bạn BẮT BUỘC phải dùng 'rag_search' tool để lấy thông tin trước khi trả lời.
- Nếu câu hỏi về thời tiết, bạn BẮT BUỘC phải dùng 'get_weather_forecast' tool.
- Tuyệt đối KHÔNG được bịa nguồn tham khảo. Nếu tool trả về rỗng, hãy nói rõ là bạn không tìm thấy thông tin.
- Luôn trích dẫn nguồn bằng [1], [2] theo output từ 'rag_search' tool (nếu có).
- Cấm tuyệt đối các ký hiệu Markdown: **, *, #, -, ~, 1.
- Nếu cần nhấn mạnh, chỉ được dùng emoji + chữ thường/hoa.
- Không sinh ra bất kỳ ký hiệu định dạng khác ngoài emoji.
- Khi viết câu trả lời cuối cùng (**Final Answer**):
  * KHÔNG sử dụng Markdown như #, *, ~, - ...
  * Trình bày văn bản rõ ràng, đẹp mắt, chuyên nghiệp.
  * Dùng emoji để nhấn mạnh cấu trúc và ý chính (ví dụ: 🌱, 🍅, 🥦, 🐛, 🌧️, 🌞, 🌬️).
  * Có thể dùng xuống dòng + emoji để tạo mục lục, tiêu đề phụ, bullet points.

Previous conversation history:
{chat_history}

### INSTRUCTION FORMAT

Question: {input}
Thought: The user's question is about... I should use the tool '...' to answer it.
Action: the action to take, should be one of [{tool_names}]
Action Input: the input to the action
Observation: the result of the action
... (this Thought/Action/Action Input/Observation có thể lặp lại N lần)
Thought: I now have enough information to answer the user's question.
Final Answer: Câu trả lời cuối cùng cho câu hỏi ban đầu, được trình bày đầy đủ như với answer và được trình bày sạch sẽ, chỉ dùng emoji để định dạng, không có Markdown.

Begin!

Question: {input}
{agent_scratchpad}
***

```

Hình 27: Mã nguồn prompt cho Tác tử ReAct

Thành phần trong prompt	Mục đích và vai trò
Personal và Goal (Bạn là AgriBot...)	Xác định vai trò: Gán cho LLM một danh tính (“AgriBot”) và mục tiêu rõ ràng (“cung cấp thông tin chính xác...”). Điều này định hướng phong cách và phạm vi hoạt động tác tử.
Tool Definitions (TOOLS YOU CAN USE: {tools})	Khai báo năng lực: Đây là một placeholder (<i>{tools}</i>) nơi LangChain sẽ tự động chèn vào danh sách và mô tả các công cụ mà tác tử có thể sử dụng. Điều này cho phép LLM “biết” mình có thể thực hiện những hành động nào.
Policy (POLICY (read carefully): ...)	Thiết lập quy tắc cứng: Đây là phần quan trọng nhất để kiểm soát hành vi. Nó đưa ra các mệnh lệnh bắt buộc (BẮT BUỘC phải dùng...), các điều cấm (Tuyệt đối KHÔNG được bịa...), và các quy định về định dạng đầu ra (Cấm ký hiệu Markdown, Dùng emoji...). Những quy tắc này đảm bảo tính nhất quán và độ tin cậy của tác tử.
Context (Previous conversation history: {chat_history})	Cung cấp bộ nhớ: Placeholder {chat_history} cho phép LangChain đưa vào lịch sử của cuộc trò chuyện, giúp tác tử hiểu được các câu hỏi nối tiếp và duy trì ngữ cảnh.

Bảng 12: Phân tích chi tiết các thành phần trong prompt

3.7.4. Kết quả thực hiện

Câu hỏi đầu vào: “Bệnh đạo ôn trên lúa có những triệu chứng gì và thời tiết ở Cần Thơ ngày mai thế nào?”

Luồng hoạt động của hệ thống:

Sau khi nhận được câu hỏi từ Người dùng, **Rewriter Chain** được kích hoạt:

--- Original Question: 'Bệnh đạo ôn trên lúa có những triệu chứng gì và thời tiết ở Cần Thơ ngày mai thế nào?' ---

--- Rewritten Question: 'Bệnh đạo ôn trên lúa có những triệu chứng gì và thời tiết ở Cần Thơ ngày mai như thế nào?' ---

Sau đó, Router Chain nhận được câu hỏi đã viết lại, dựa trên nội dung có cả yếu tố “*bệnh nông nghiệp*” và “*thời tiết*” mà chuỗi này phân loại ý định của người dùng.

--- Intent Classified As: TOOL_USER ---

Sau khi đã xác định được ý định người dùng là gì, hệ thống tiếp đến sẽ thực thi vòng lặp **ReAct**:

Vòng lặp 1 – Xử lý phần bệnh đạo ôn

Tác tử AI (mô hình Gemini) phân tích câu hỏi phức hợp và lập kế hoạch hành động. Nó nhận diện hai phần riêng biệt: thông tin về bệnh và thông tin thời tiết. Tác tử quyết định sẽ sử dụng công cụ ‘*rag_search*’ để tìm kiếm thông tin về bệnh đạo ôn trước.

Action: *rag_search*

Action Input: Bệnh đạo ôn trên lúa có những triệu chứng gì?

```
{"answer": "Tôi là AgriBot, một trợ lý AI chuyên về nông nghiệp. Thông tin về triệu chứng, cách phòng ngừa và điều trị bệnh đạo ôn trên lúa chưa có trong kho tài liệu của tôi! Tài liệu hiện có chỉ đề cập đến các bệnh khác trên cây tiêu và một số biện pháp phòng trừ sâu bệnh hại trên lúa sau bão, bao gồm cả bệnh bạc lá lúa [2], nhưng không cung cấp thông tin cụ thể về bệnh đạo ôn trên lúa.", "sources": [{"start": 0, "end": 200, "text": "ngap_sau_mua_bao (ngap_sau_mua_bao.pdf)"}], "timing": {"retrieve": 18.01866626739502, "generate": 1.1324098110198975}}
```

Vòng lặp 2 – Xử lý phần thời tiết

Tác tử ghi nhận kết quả từ ‘*rag_search*’ (không tìm thấy thông tin về bệnh đạo ôn). Dựa trên kế hoạch ban đầu, tác tử tiếp tục xử lý phần còn lại của câu hỏi là dự báo thời tiết. Nó quyết định sử dụng công cụ ‘*get_weather_forecast*’.

Action: get_weather_forecast

Action Input: Cần Thơ 🌤️ Dự báo thời tiết tại Cần Thơ:

📅 Hôm nay (20/09):

- Tình hình: Mây cụm
- Nhiệt độ: Khoảng 31.8°C
- Khả năng mưa: 0%

📅 Ngày mai (21/09):

- Tình hình: Mây rải rác
- Nhiệt độ: Khoảng 31.3°C
- Khả năng mưa: 0%

Tổng hợp và đưa ra Câu trả lời cuối cùng

Tác tử nhận thấy đã thu thập đủ thông tin cho cả hai phần của câu hỏi, tác tử quyết định tổng hợp thông tin thời tiết và thông báo về bệnh đạo ôn để đưa ra '*Final Answer*'. Tác tử tạo ra một câu trả lời hoàn chỉnh, kết hợp cả hai thông tin theo đúng định dạng được yêu cầu trong prompt.

Thought: I now have enough information to answer the user's question. I will combine the weather forecast and the response from the RAG search.

Final Answer: 🌿 Thông tin về bệnh đạo ôn và thời tiết Cần Thơ:

🌱 Bệnh đạo ôn trên lúa: Rất tiếc, tôi chưa tìm thấy thông tin về triệu chứng của bệnh đạo ôn trên lúa trong kho dữ liệu của mình. ⚠️ Tôi chỉ có thông tin về các bệnh khác trên cây trồng khác và một số biện pháp phòng trừ sâu bệnh hại trên lúa sau bão [2].

🌞 Dự báo thời tiết Cần Thơ:

📅 Ngày mai (21/09):

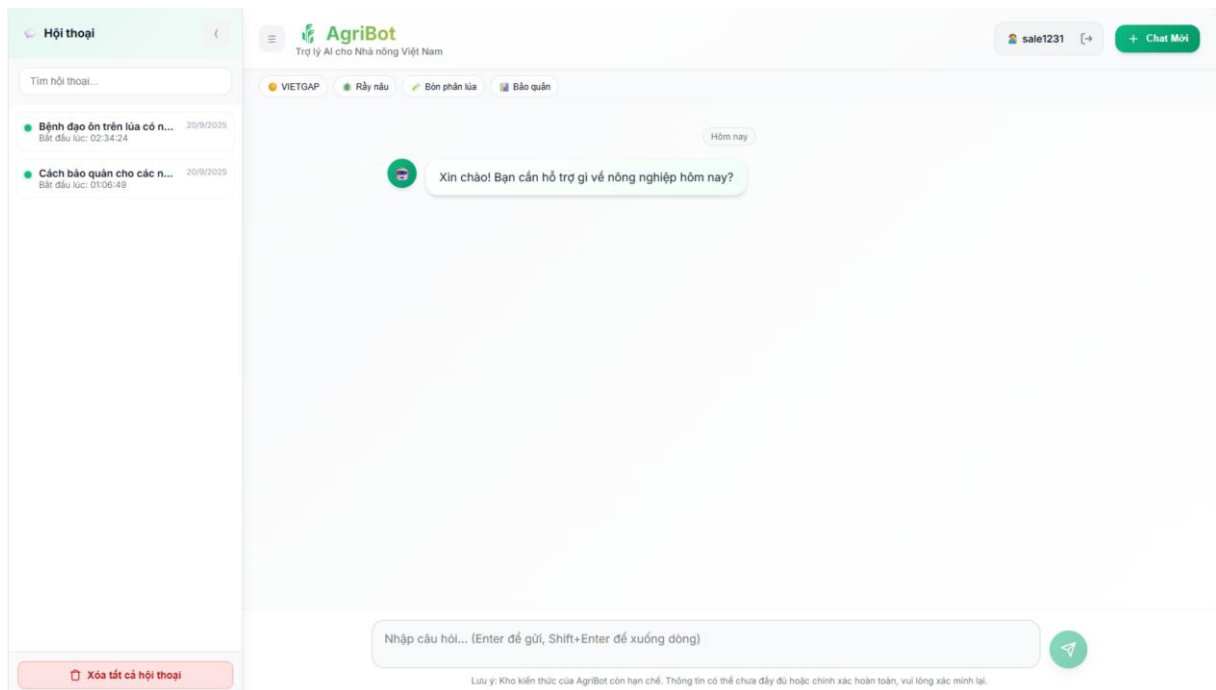
- Trời ☁️ mây rải rác.
- Nhiệt độ khoảng 🌡️ 31.3°C.
- Khả năng mưa 🌧️: 0%

[2] ngap_sau_mua_bao (ngap_sau_mua_bao.pdf)

Kết thúc chuỗi hoạt động

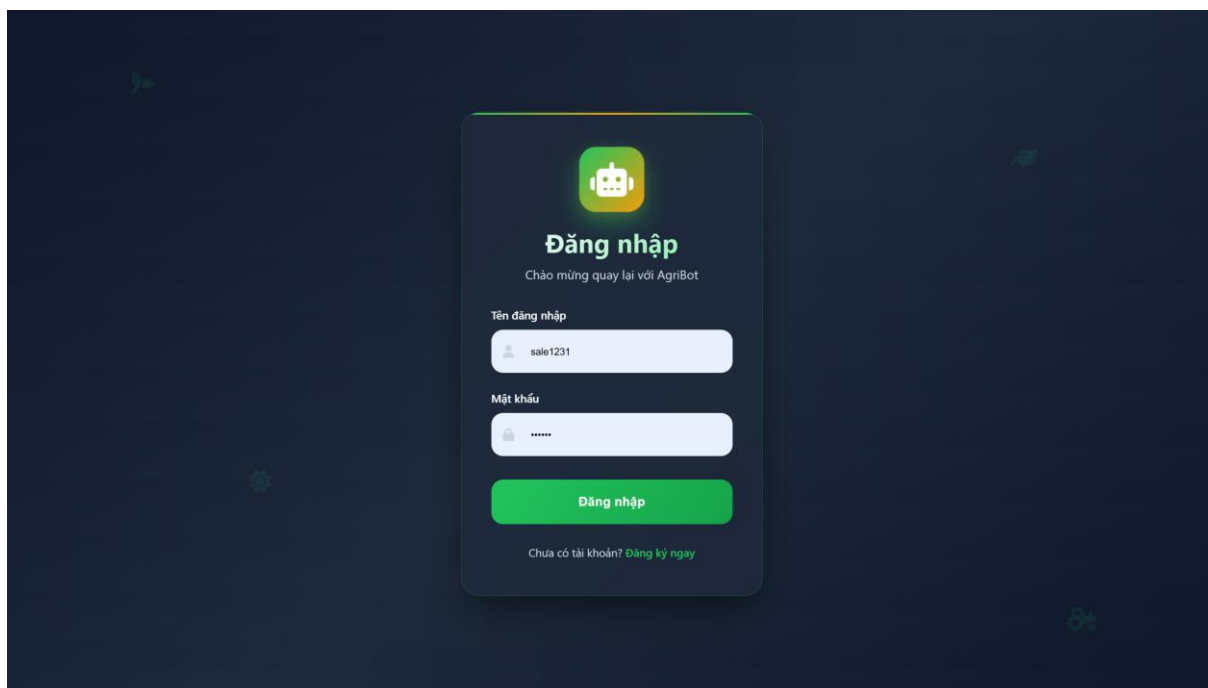
3.8. Thiết kế giao diện người dùng (User Interface – UI)

3.8.1. Giao diện trang chat chính



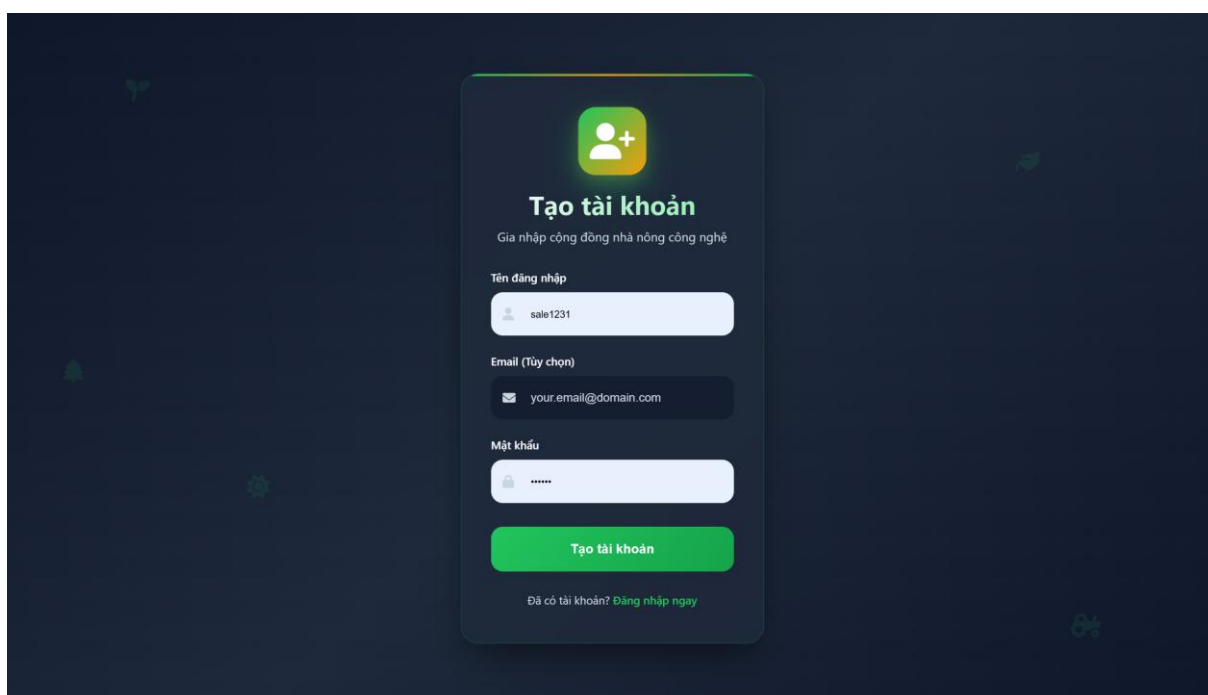
Hình 28: Giao diện chính của hệ thống

3.8.2. Giao diện trang xác thực đăng nhập



Hình 29: Giao diện trang đăng nhập của hệ thống

3.8.3. Giao diện trang xác thực đăng ký

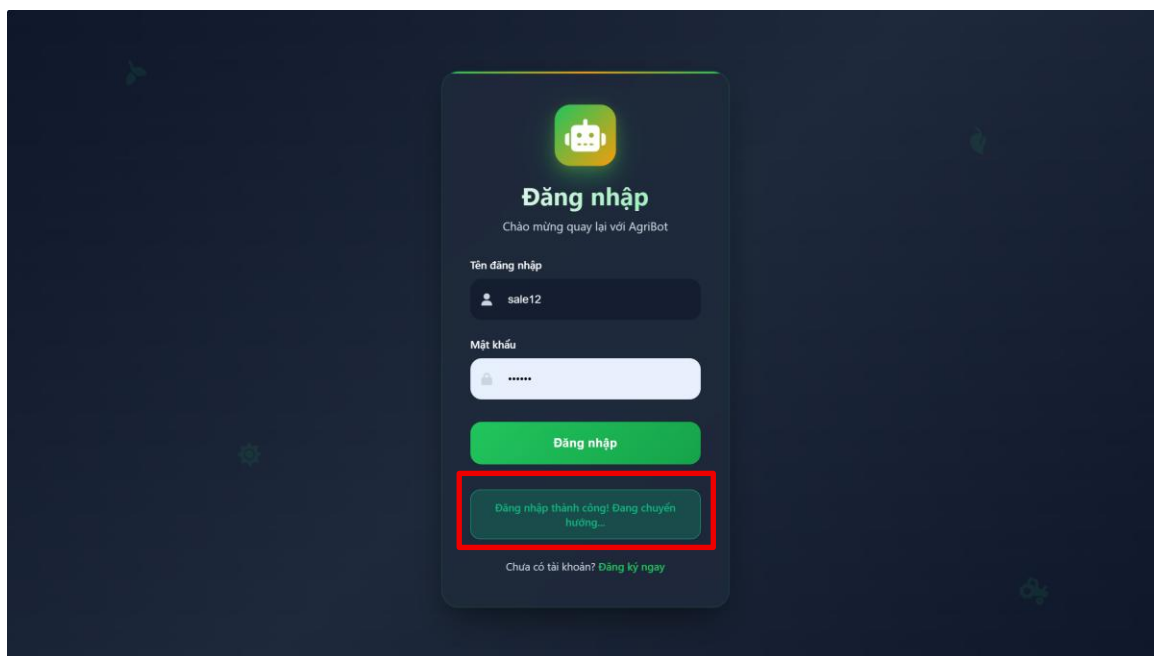


Hình 30: Giao diện trang đăng ký tài khoản

3.8.4. Giao diện khi thực hiện các chức năng chính của hệ thống

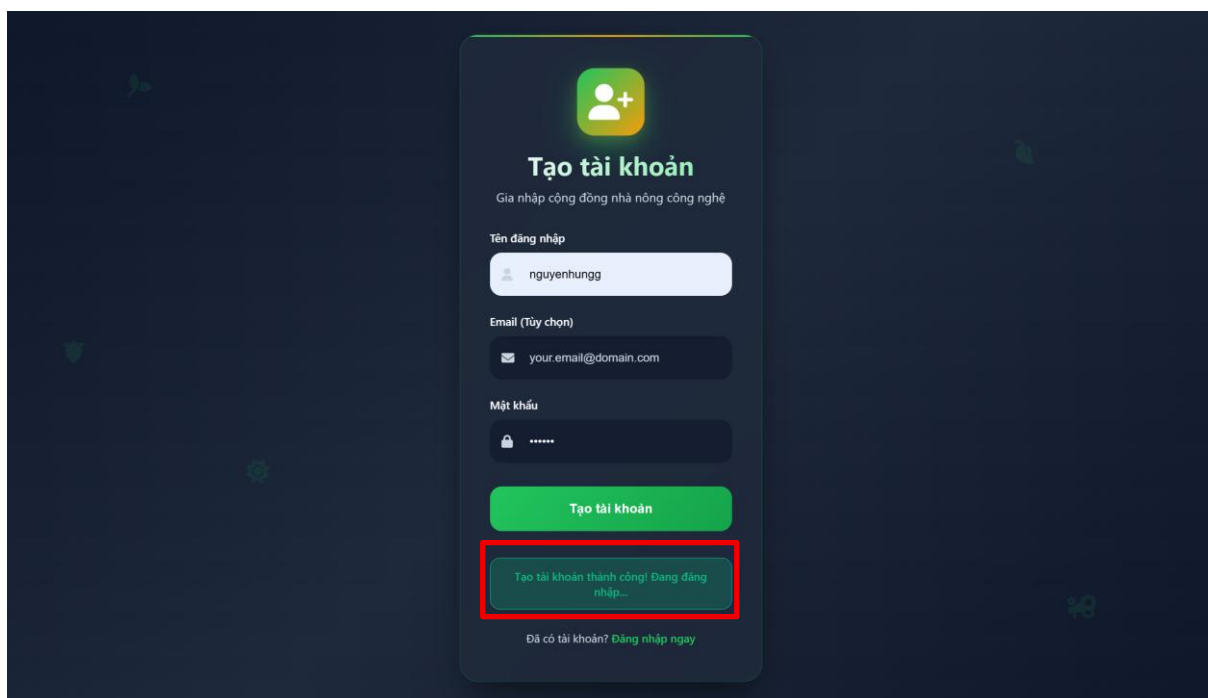
3.8.4.1. Giao diện khi thực hiện đăng nhập thành công

Khi người dùng nhập tên đăng nhập và mật khẩu thành công, hệ thống sẽ xuất hiện thông báo đăng nhập thành công và chuyển hướng đến trang chat chính.



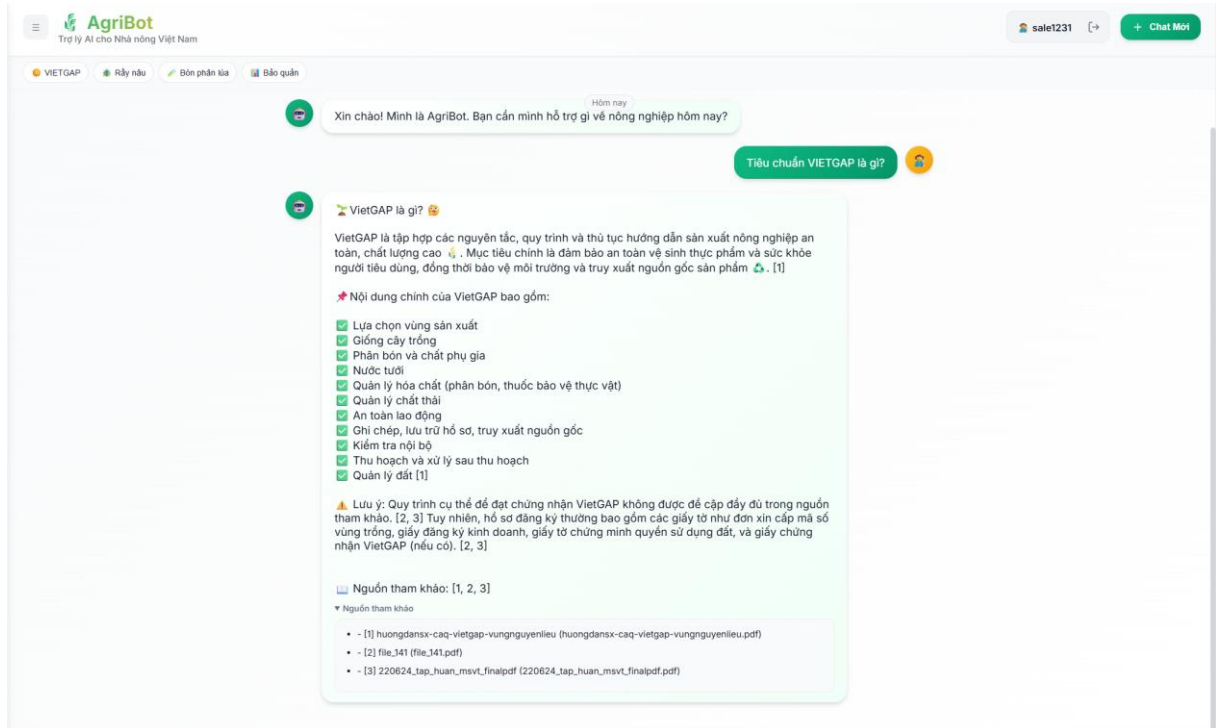
Hình 31: Giao diện hệ thống khi người dùng đăng nhập thành công

3.8.4.2. Giao diện khi thực hiện đăng ký thành công



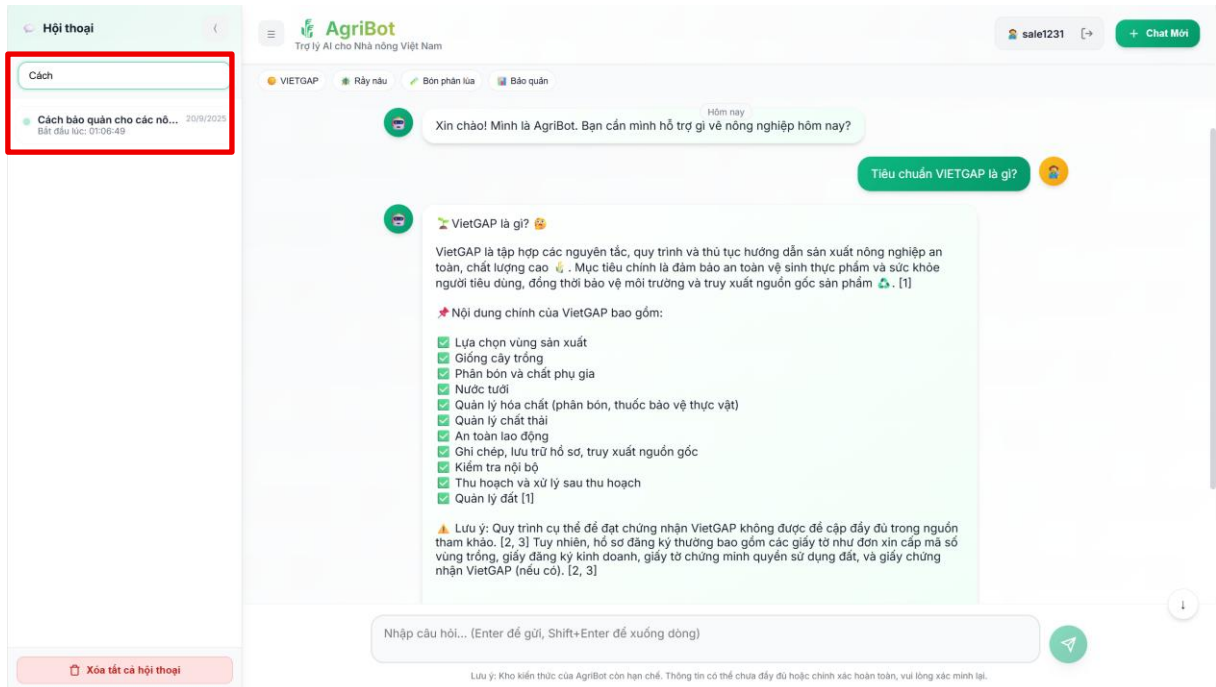
Hình 32: Giao diện hệ thống khi người dùng đăng ký thành công

3.8.4.3. Giao diện khi thực hiện hội thoại với hệ thống



Hình 33: Giao diện hỏi – đáp của hệ thống

3.8.4.4. Giao diện khi thực hiện chức năng tìm kiếm hội thoại



Hình 34: Giao diện khi Người dùng thực hiện tìm kiếm hội thoại

Chương 4. KẾT LUẬN VÀ XÂY DỰNG HƯỚNG PHÁT TRIỂN

4.1. Kết luận

Trong bối cảnh ngành nông nghiệp đang chuyển đổi số mạnh mẽ, đề tài “**Ứng dụng RAG xây dựng hệ thống tác tử AI trong nông nghiệp thông minh**” đã thành công trong việc xây dựng và triển khai một hệ thống tích hợp hiệu quả mô hình LLM của Google Gemini với một cơ sở tri thức nông nghiệp chuyên sâu.

Kiến trúc tác tử ReAct đã được áp dụng thành công, giúp hệ thống thể hiện khả năng suy luận, lập kế hoạch và thực thi hành động một cách linh hoạt. Các chuỗi xử lý như Rewriter Chain và Router Chain đã tối ưu hóa luồng xử lý câu hỏi, đảm bảo hệ thống phản hồi nhanh chóng và chính xác, đồng thời tiết kiệm tài nguyên. Hệ thống đã chứng minh được tiềm năng ứng dụng trong việc cung cấp thông tin kịp thời, góp phần nâng cao năng suất và hiệu quả quản lý cho người nông dân.

4.2. Những điều đã đạt được và điều chưa thực hiện

4.2.1. Những điều đạt được

Qua quá trình nghiên cứu và phát triển, đề tài đã đạt được những kết quả đáng kể sau:

Xây dựng thành công hệ thống AI Agent.



Tích hợp hiệu quả kỹ thuật Retrieval-Augmented Generation(RAG)

Triển khai kiến trúc tác tử ReAct

Phát triển các chuỗi xử lý thông minh

Thiết kế hệ thống theo hướng Module

Tạo một giao diện người dùng thân thiện

4.2.2. Những hạn chế và hướng phát triển

4.2.2.1. Những hạn chế

Phạm vi cơ sở tri thức còn giới hạn: Hiện tại, cơ sở tri thức của hệ thống chưa thực sự đủ bao quát cho ngành Nông nghiệp Việt Nam. Thông tin về các loại cây trồng, kỹ thuật canh tác đặc thù hoặc các vùng miền cụ thể vẫn còn thiếu.

Khả năng xử lý đa phương tiện: Hệ thống hiện tại chủ yếu xử lý đầu vào và đầu ra dạng văn bản. Khả năng nhận diện hình ảnh hoặc xử lý giọng nói chưa được tích hợp.

Thời gian: Đối với các câu hỏi phức tạp yêu cầu nhiều vòng lặp và truy vấn công cụ, thời gian phản hồi chờ của hệ thống đôi khi còn tương đối dài, ảnh hưởng đến trải nghiệm người dùng.

Tính năng cá nhân hóa: Hệ thống chưa có khả năng cá nhân hóa lời khuyên dựa trên hồ sơ cụ thể của người nông dân.

4.2.2.2. Hướng phát triển trong tương lai

Khắc phục những điểm hạn chế mà đề tài đang gặp phải.

Phát triển module Quản lý Tri thức cho Quản trị viên: Xây dựng một giao diện quản trị web cho phép quản trị viên có khả năng tải lên các tài liệu mới, xóa các tài liệu không còn phù hợp và kích hoạt quá trình xử lý dữ liệu trực tiếp từ giao diện web nhằm hạn chế sự phụ thuộc vào việc can thiệp mã nguồn.

Phát triển module Chuẩn đoán bệnh qua hình ảnh.

Phát triển Cộng đồng Người dùng (diễn đàn nhỏ)



TÀI LIỆU THAM KHẢO

- [1] Ngocdiep. (2024, December 30). *Trí tuệ nhân tạo tạo sinh*  *khái niệm, đặc điểm và ứng dụng*. VinUni. <https://vinuni.edu.vn/vi/tri-tue-nhan-cao-tao-sinh-khai-niem-dac-diem-va-ung-dung/>
- [2] GeeksforGeeks. (2025, September 18). *What is a Large Language Model (LLM)*. GeeksforGeeks. <https://www.geeksforgeeks.org/artificial-intelligence/large-language-model-llm/>
- [3] GeeksforGeeks. (2025a, July 23). *What are Vector Embeddings?* GeeksforGeeks. <https://www.geeksforgeeks.org/nlp/what-are-vector-embeddings/>
- [4] GeeksforGeeks. (2025b, August 21). *What is RetrievalAugmented Generation (RAG) ?* GeeksforGeeks. <https://www.geeksforgeeks.org/nlp/what-is-retrieval-augmented-generation-rag/>
- [5] GeeksforGeeks. (2025c, September 3). *What is Generative AI?* GeeksforGeeks. <https://www.geeksforgeeks.org/artificial-intelligence/what-is-generative-ai/>
- [6] Lettner, F., & Reitbauer, A. (2025, April 8). Dynatrace accelerates business transformation with new AI observability solution. *Dynatrace News*. <https://www.dynatrace.com/news/blog/dynatrace-accelerates-business-transformation-with-new-ai-observability-solution/>
- [7] *LangChain*. (n.d.). <https://www.langchain.com/>
- [8] *ReAct prompting – Nextra*. (n.d.). <https://www.promptingguide.ai/techniques/react>
- [9] GeeksforGeeks. (2025b, August 14). *Agents in AI*. GeeksforGeeks. <https://www.geeksforgeeks.org/artificial-intelligence/agents-artificial-intelligence/>
- [10] *Vector stores* |  *LangChain*. (n.d.). <https://python.langchain.com/docs/concepts/vectorstores/>
- [11] GeeksforGeeks. (2025d, August 25). *Introduction to LangChain*. GeeksforGeeks. <https://www.geeksforgeeks.org/artificial-intelligence/introduction-to-langchain/>
- [12] OpenWeatherMap.org. (n.d.). *Weather API - OpenWeatherMap*. <http://api.openweathermap.org/>

